MiniExcel

Public

1 Branch 91 Tags Go to file t Go to file Add file Code

michelebastione Merge PR 785: Fixes enum behaviour and adds support for D...

9d59b57 · 6 hours ago

.github	Added benchmark workflow file	2 weeks ago
benchmarks/MiniExcel.Be...	Benchmarks refactoring and ada...	2 weeks ago
docs	1.41.1	3 weeks ago
samples	Fixed issue 527	18 hours ago
src/MiniExcel	Minor refactoring and code clea...	18 hours ago
tests	Fixed issue 527	18 hours ago
.editorconfig	CSV enumeration and code reus...	last year
.gitattributes	Make System.DateOnly available...	last year
.gitignore	Benchmarks refactoring and ada...	2 weeks ago
LICENSE	Update license	4 years ago
MiniExcel.sln	Benchmarks refactoring and ada...	2 weeks ago
README-NuGet.md	[doc] origin path to mini-softwa...	last month

About

Fast, Low-Memory, Easy Excel .NET helper to import/export/template spreadsheet (support Linux, Mac)

www.nuget.org/packages/MiniExc...

#csv #excel #xlsx #openxml #excel-report #miniexcel

Readme

Apache-2.0 license

Activity

Custom properties

3k stars

37 watching

361 forks

Report repository

Releases 89

1.41.1 Latest

3 weeks ago

+ 88 releases

README.md	Update README.md	10 hours ago
README.zh-CN.md	add deepwiki link and badge.	yesterday
README.zh-Hant.md	add deepwiki link and badge.	yesterday
appveyor.yml	Fix CI error CS0122: 'DateTimeH...	4 years ago

README | Apache-2.0 license



This project is part of the .NET Foundation and operates under their code of conduct.

English | 简体中文 | 繁體中文

Your Star and Donate can make MiniExcel better

Introduction

MiniExcel is simple and efficient to avoid OOM's .NET processing Excel tool.

At present, most popular frameworks need to load all the data into the memory to facilitate operation, but it will cause memory consumption problems. MiniExcel tries to use algorithm from a stream to reduce the original 1000 MB occupation to a few MB to avoid OOM(out of memory).

Sponsor this project

https://miniexcel.github.io

Used by 812

Contributors 59

+ 45 contributors

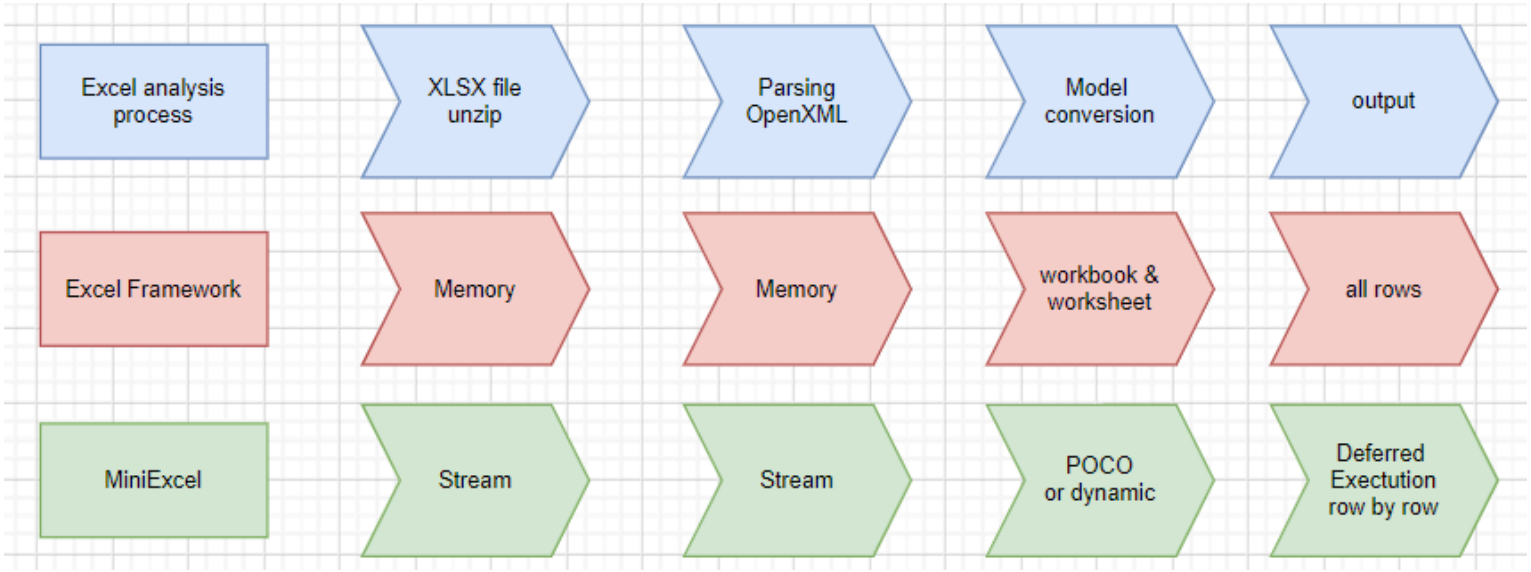
Deployments 16

github-pages last year

+ 15 deployments

Languages

C# 99.9% Other 0.1%



Features

- Low memory consumption, avoid OOM (out of memory) and full GC
- Support `real-time` operation of each row of data
- Support LINQ deferred execution, it can do low-consumption, fast paging and other complex queries
- Lightweight, without Microsoft Office installed, no COM+, DLL size is less than 150KB
- Easy API style to read/write/fill excel

Get Started

- Import/Query Excel
- Export/Create Excel
- Excel Template
- Excel Column Name/Index/Ignore Attribute
- Examples

Installation

You can install the package from NuGet

Release Notes

Please Check Release Notes

TODO

Please Check TODO

Performance

Benchmarks logic can be found in MiniExcel.Benchmarks , and test cli

```
dotnet run -p .\benchmarks\MiniExcel.Benchmarks\ -c Release -f netcoreapp3.1 -- -f * --join
```

Output from the latest run is :

```
BenchmarkDotNet=v0.12.1, OS=Windows 10.0.19042
Intel Core i7-7700 CPU 3.60GHz (Kaby Lake), 1 CPU, 8 logical and 4 physical cores
  [Host]     : .NET Framework 4.8 (4.8.4341.0), X64 RyuJIT
  Job-ZYYABG : .NET Framework 4.8 (4.8.4341.0), X64 RyuJIT
IterationCount=3  LaunchCount=3  WarmupCount=3
```

Benchmark History : Link

Import/Query Excel

Logic : **Test1,000,000x10.xlsx** as performance test basic file, 1,000,000 rows * 10 columns "HelloWorld" cells, 23 MB file size

Library	Method	Max Memory Usage	Mean
MiniExcel	'MiniExcel QueryFirst'	0.109 MB	0.0007264 sec
ExcelDataReader	'ExcelDataReader QueryFirst'	15.24 MB	10.66421 sec

Library	Method	Max Memory Usage	Mean
MiniExcel	'MiniExcel Query'	17.3 MB	14.17933 sec
ExcelDataReader	'ExcelDataReader Query'	17.3 MB	22.56508 sec
Epplus	'Epplus QueryFirst'	1,452 MB	18.19801 sec
Epplus	'Epplus Query'	1,451 MB	23.64747 sec
OpenXmlSDK	'OpenXmlSDK Query'	1,412 MB	52.00327 sec
OpenXmlSDK	'OpenXmlSDK QueryFirst'	1,413 MB	52.34865 sec
ClosedXml	'ClosedXml QueryFirst'	2,158 MB	66.18897 sec
ClosedXml	'ClosedXml Query'	2,184 MB	191.43412 sec

Export/Create Excel

Logic : create a total of 10,000,000 "HelloWorld" excel

Library	Method	Max Memory Usage	Mean
MiniExcel	'MiniExcel Create Xlsx'	15 MB	11.53181 sec
Epplus	'Epplus Create Xlsx'	1,204 MB	22.50971 sec
OpenXmlSdk	'OpenXmlSdk Create Xlsx'	2,621 MB	42.47399 sec
ClosedXml	'ClosedXml Create Xlsx'	7,141 MB	140.93992 sec

Excel Query/Import

1. Execute a query and map the results to a strongly typed IEnumerable [Try it]

Recommand to use Stream.Query because of better efficiency.

```
public class UserAccount
{
```

```
    public Guid ID { get; set; }
    public string Name { get; set; }
    public DateTime BoD { get; set; }
    public int Age { get; set; }
    public bool VIP { get; set; }
    public decimal Points { get; set; }
}

var rows = MiniExcel.Query<UserAccount>(path);

// or

using (var stream = File.OpenRead(path))
    var rows = stream.Query<UserAccount>();
```

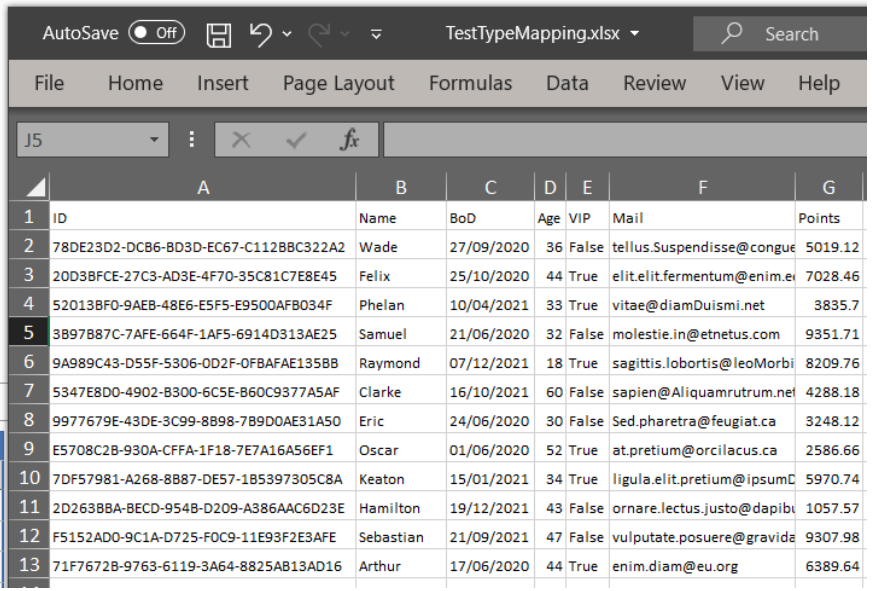
```
void Main()
{
    using (var stream = File.OpenRead(@"D:\git\MiniExcel\samples\xlsx\TestTypeMapping.xlsx"))
    {
        var rows = stream.Query<UserAccount>().ToList();
        Console.WriteLine(rows);
    }
}

public class UserAccount
{
    public Guid ID { get; set; }
    public string Name { get; set; }
    public DateTime BoD { get; set; }
    public int Age { get; set; }
    public bool VIP { get; set; }
    public decimal Points { get; set; }
}
```

Results λ SQL IL Tree

List<UserAccount> (100 items)

ID	Name	BoD	Age	VIP	Points
78de23d2-dcb6-bd3d-ec67-c112bbc322a2	Wade	2020-09-27 12:00:00 AM	36	False	5019.12
20d3bfce-27c3-ad3e-4f70-35c81c7e8e45	Felix	2020-10-25 12:00:00 AM	44	True	7028.46
52013bf0-9aeb-48e6-e5f5-e9500afb034f	Phelan	2021-10-04 12:00:00 AM	33	True	3835.7



	A	B	C	D	E	F	G
1	ID	Name	BoD	Age	VIP	Mail	Points
2	78DE23D2-DC86-BD3D-EC67-C112BBC322A2	Wade	27/09/2020	36	False	tellus.Suspendisse@congue	5019.12
3	20D3BFCE-27C3-AD3E-4F70-35C81C7E8E45	Felix	25/10/2020	44	True	elit.elit.fermentum@enim.e	7028.46
4	52013BF0-9AEB-48E6-E5F5-E9500AFB034F	Phelan	10/04/2021	33	True	vitae@diamDuismi.net	3835.7
5	3B97B87C-7AFE-664F-1AF5-6914D313AE25	Samuel	21/06/2020	32	False	molestie.in@etnetus.com	9351.71
6	9A989C43-D55F-5306-0D2F-0FBAFAE135BB	Raymond	07/12/2021	18	True	sagittis.lobortis@leoMorbi	8209.76
7	5347E8D0-4902-B300-6C5E-B60C9377A5AF	Clarke	16/10/2021	60	False	sapien@Aliquamrutrum.net	4288.18
8	9977679E-43DE-3C99-8B98-7B9D0AE31A50	Eric	24/06/2020	30	False	Sed.pharetra@feugiat.ca	3248.12
9	E5708C2B-930A-CFFA-1F18-7E7A16A56EF1	Oscar	01/06/2020	52	True	at.pretium@orcilacus.ca	2586.66
10	7DF57981-A268-8B87-DE57-1B5397305C8A	Keaton	15/01/2021	34	True	ligula.elit.pretium@ipsumD	5970.74
11	2D263BBA-BECD-954B-D209-A386AAC6D23E	Hamilton	19/12/2021	43	False	ornare.lectus.justo@dapibu	1057.57
12	F5152AD0-9C1A-D725-F0C9-11E93F2E3AFE	Sebastian	21/09/2021	47	False	vulputate.posuere@gravida	9307.98
13	71F7672B-9763-6119-3A64-8825AB13AD16	Arthur	17/06/2020	44	True	enim.diam@eu.org	6389.64

2. Execute a query and map it to a list of dynamic objects without using head [Try it]

- dynamic key is `A.B.C.D.`

MiniExcel	1
Github	2

```
var rows = MiniExcel.Query(path).ToList();

// or
using (var stream = File.OpenRead(path))
{
    var rows = stream.Query().ToList();

    Assert.Equal("MiniExcel", rows[0].A);
    Assert.Equal(1, rows[0].B);
    Assert.Equal("Github", rows[1].A);
    Assert.Equal(2, rows[1].B);
}
```

3. Execute a query with first header row [Try it]

note : same column name use last right one

Input Excel :

Column1	Column2
MiniExcel	1
Github	2

```
var rows = MiniExcel.Query(useHeaderRow:true).ToList();

// or

using (var stream = File.OpenRead(path))
{
    var rows = stream.Query(useHeaderRow:true).ToList();

    Assert.Equal("MiniExcel", rows[0].Column1);
```

```
    Assert.Equal(1, rows[0].Column2);
    Assert.Equal("Github", rows[1].Column1);
    Assert.Equal(2, rows[1].Column2);
}
```

4. Query Support LINQ Extension First/Take/Skip ...etc

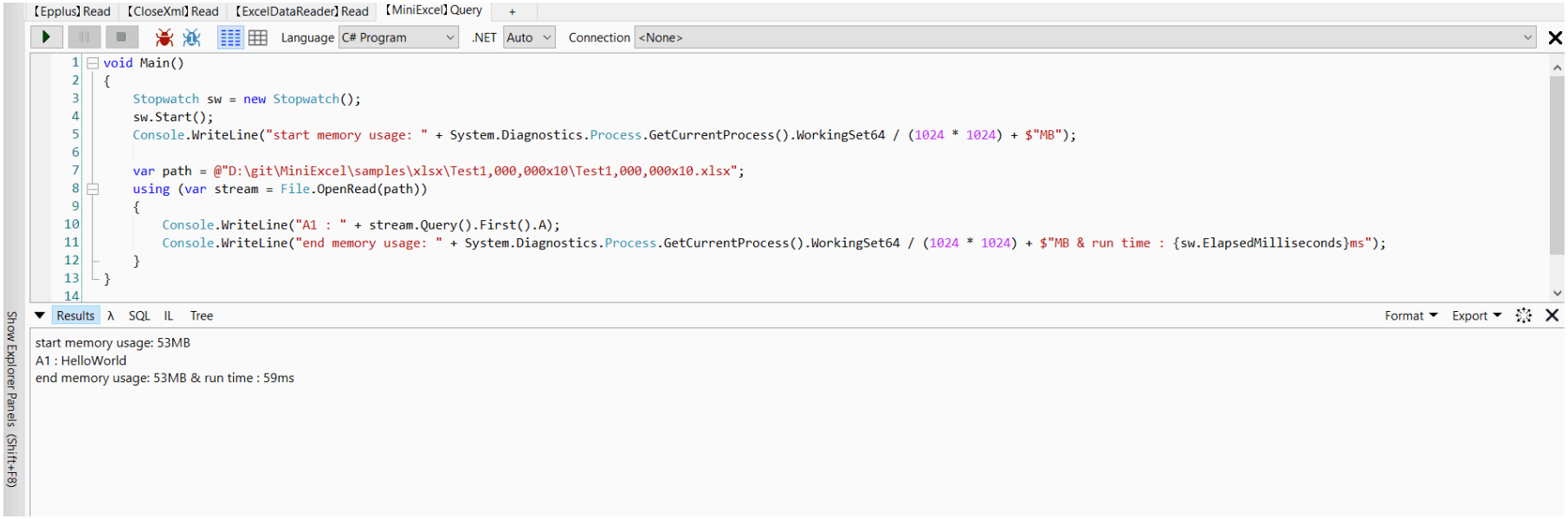
Query First

```
var row = MiniExcel.Query(path).First();
Assert.Equal("HelloWorld", row.A);

// or

using (var stream = File.OpenRead(path))
{
    var row = stream.Query().First();
    Assert.Equal("HelloWorld", row.A);
}
```

Performance between MiniExcel/ExcelDataReader/ClosedXML/EPPlus



5. Query by sheet name

```
MiniExcel.Query(path, sheetName: "SheetName");
//or
stream.Query(sheetName: "SheetName");
```

6. Query all sheet name and rows

```
var sheetNames = MiniExcel.GetSheetNames(path);
foreach (var sheetName in sheetNames)
{
    var rows = MiniExcel.Query(path, sheetName: sheetName);
}
```

7. Get Columns

```
var columns = MiniExcel.GetColumns(path); // e.g result : ["A","B"...]

var cnt = columns.Count;  // get column count
```

8. Dynamic Query cast row to `IDictionary<string,object>`

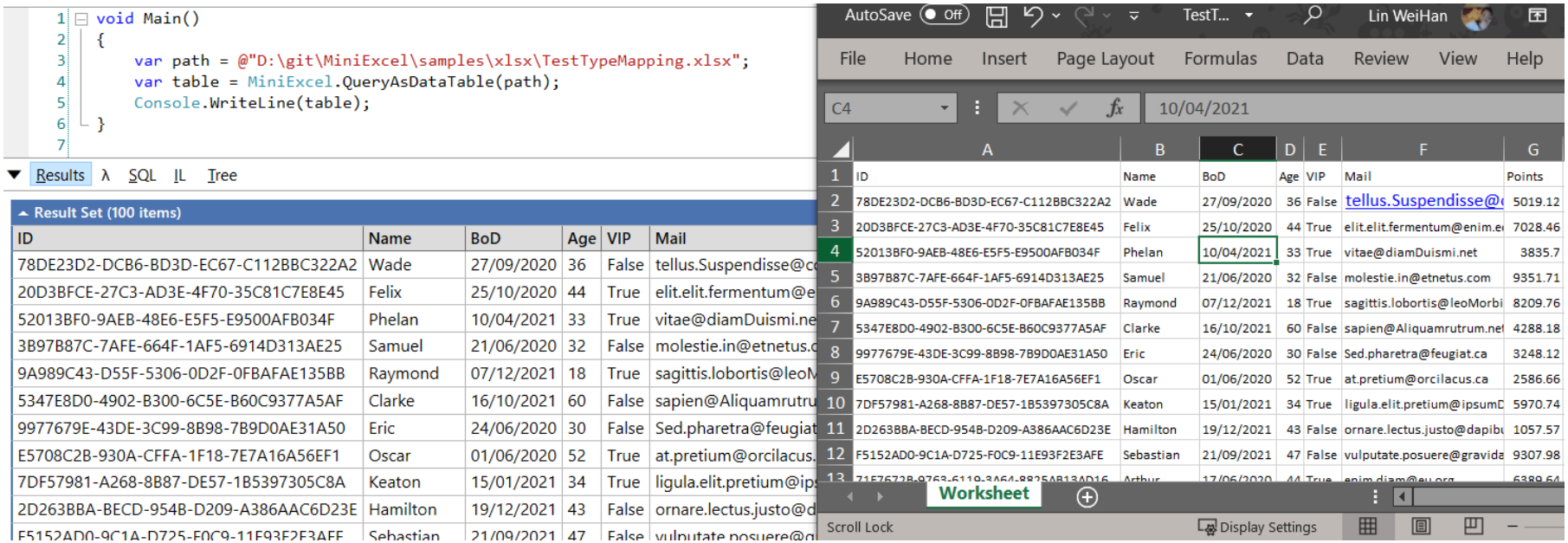
```
foreach(IDictionary<string,object> row in MiniExcel.Query(path))
{
    //..
}

// or
var rows = MiniExcel.Query(path).Cast<IDictionary<string,object>>();
// or Query specified ranges (capitalized)
// A2 represents the second row of column A, C3 represents the third row of column C
// If you don't want to restrict rows, just don't include numbers
var rows = MiniExcel.QueryRange(path, startCell: "A2", endCell: "C3").Cast<IDictionary<string, object>>();
```

9. Query Excel return DataTable

Not recommended, because DataTable will load all data into memory and lose MiniExcel's low memory consumption feature.

```
var table = MiniExcel.QueryAsDataTable(path, useHeaderRow: true);
```



```
void Main()
{
    var path = @"D:\git\MiniExcel\samples\xlsx\TestTypeMapping.xlsx";
    var table = MiniExcel.QueryAsDataTable(path);
    Console.WriteLine(table);
}
```

Results λ SQL IL Tree

Result Set (100 items)

ID	Name	BoD	Age	VIP	Mail
78DE23D2-DCB6-BD3D-EC67-C112BBC322A2	Wade	27/09/2020	36	False	tellus.Suspendisse@c
20D3BFCE-27C3-AD3E-4F70-35C81C7E8E45	Felix	25/10/2020	44	True	elit.elit.fermentum@e
52013BF0-9AEB-48E6-E5F5-E9500AFB034F	Phelan	10/04/2021	33	True	vitae@diamDuismi.ne
3B97B87C-7AFE-664F-1AF5-6914D313AE25	Samuel	21/06/2020	32	False	molestie.in@etnetus.c
9A989C43-D55F-5306-0D2F-0FBAFAE135BB	Raymond	07/12/2021	18	True	sagittis.lobortis@leoM
5347E8D0-4902-B300-6C5E-B60C9377A5AF	Clarke	16/10/2021	60	False	sapien@Aliquamrutru
9977679E-43DE-3C99-8B98-7B9D0AE31A50	Eric	24/06/2020	30	False	Sed.pharetra@feugiat
E5708C2B-930A-CFFA-1F18-7E7A16A56EF1	Oscar	01/06/2020	52	True	at.pretium@orcilacus.
7DF57981-A268-8B87-DE57-1B5397305C8A	Keaton	15/01/2021	34	True	ligula.elit.pretium@ips
2D263BBA-BECD-954B-D209-A386AAC6D23E	Hamilton	19/12/2021	43	False	ornare.lectus.justo@d
F5152AD0-9C1A-D725-F0C9-11E93F2E3AFE	Sebastian	21/09/2021	47	False	vulputate.posuere@gl

AutoSave Off TestT... Lin WeiHan

File Home Insert Page Layout Formulas Data Review View Help

C4 10/04/2021

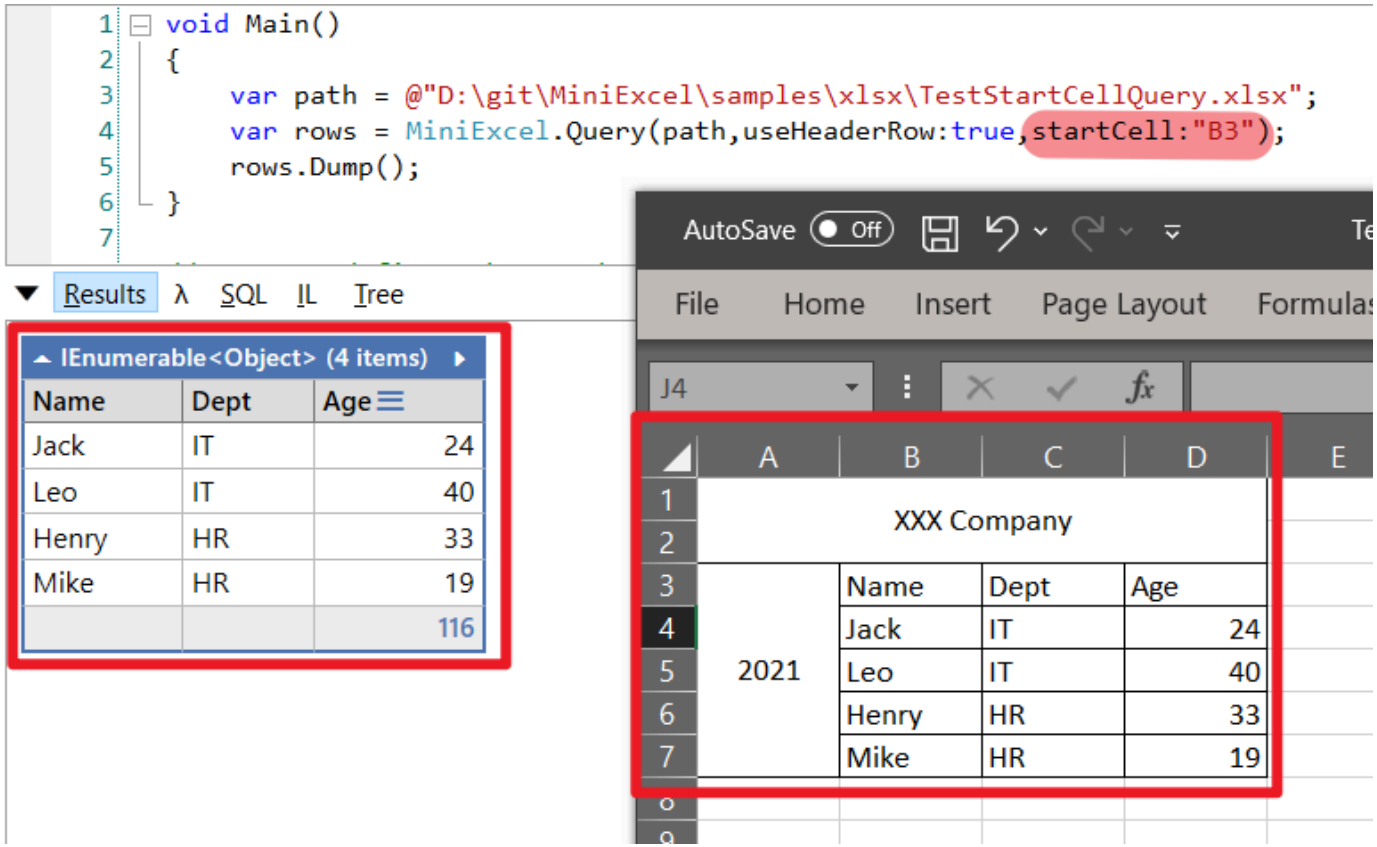
	A	B	C	D	E	F	G
1	ID	Name	BoD	Age	VIP	Mail	Points
2	78DE23D2-DCB6-BD3D-EC67-C112BBC322A2	Wade	27/09/2020	36	False	tellus.Suspendisse@	5019.12
3	20D3BFCE-27C3-AD3E-4F70-35C81C7E8E45	Felix	25/10/2020	44	True	elit.elit.fermentum@enim.e	7028.46
4	52013BF0-9AEB-48E6-E5F5-E9500AFB034F	Phelan	10/04/2021	33	True	vitae@diamDuismi.net	3835.7
5	3B97B87C-7AFE-664F-1AF5-6914D313AE25	Samuel	21/06/2020	32	False	molestie.in@etnetus.com	9351.71
6	9A989C43-D55F-5306-0D2F-0FBAFAE135BB	Raymond	07/12/2021	18	True	sagittis.lobortis@leoMorbi	8209.76
7	5347E8D0-4902-B300-6C5E-B60C9377A5AF	Clarke	16/10/2021	60	False	sapien@Aliquamrutrum.net	4288.18
8	9977679E-43DE-3C99-8B98-7B9D0AE31A50	Eric	24/06/2020	30	False	Sed.pharetra@feugiat.ca	3248.12
9	E5708C2B-930A-CFFA-1F18-7E7A16A56EF1	Oscar	01/06/2020	52	True	at.pretium@orcilacus.ca	2586.66
10	7DF57981-A268-8B87-DE57-1B5397305C8A	Keaton	15/01/2021	34	True	ligula.elit.pretium@ipsumD	5970.74
11	2D263BBA-BECD-954B-D209-A386AAC6D23E	Hamilton	19/12/2021	43	False	ornare.lectus.justo@dapibu	1057.57
12	F5152AD0-9C1A-D725-F0C9-11E93F2E3AFE	Sebastian	21/09/2021	47	False	vulputate.posuere@gravida	9307.98
13	71E7672B-9763-6119-3A64-8825AB13AD16	Arthur	17/06/2020	44	True	enim.diam@eu.org	6389.64

Worksheet

Scroll Lock Display Settings

10. Specify the cell to start reading data

```
MiniExcel.Query(path,useHeaderRow:true,startCell:"B3")
```

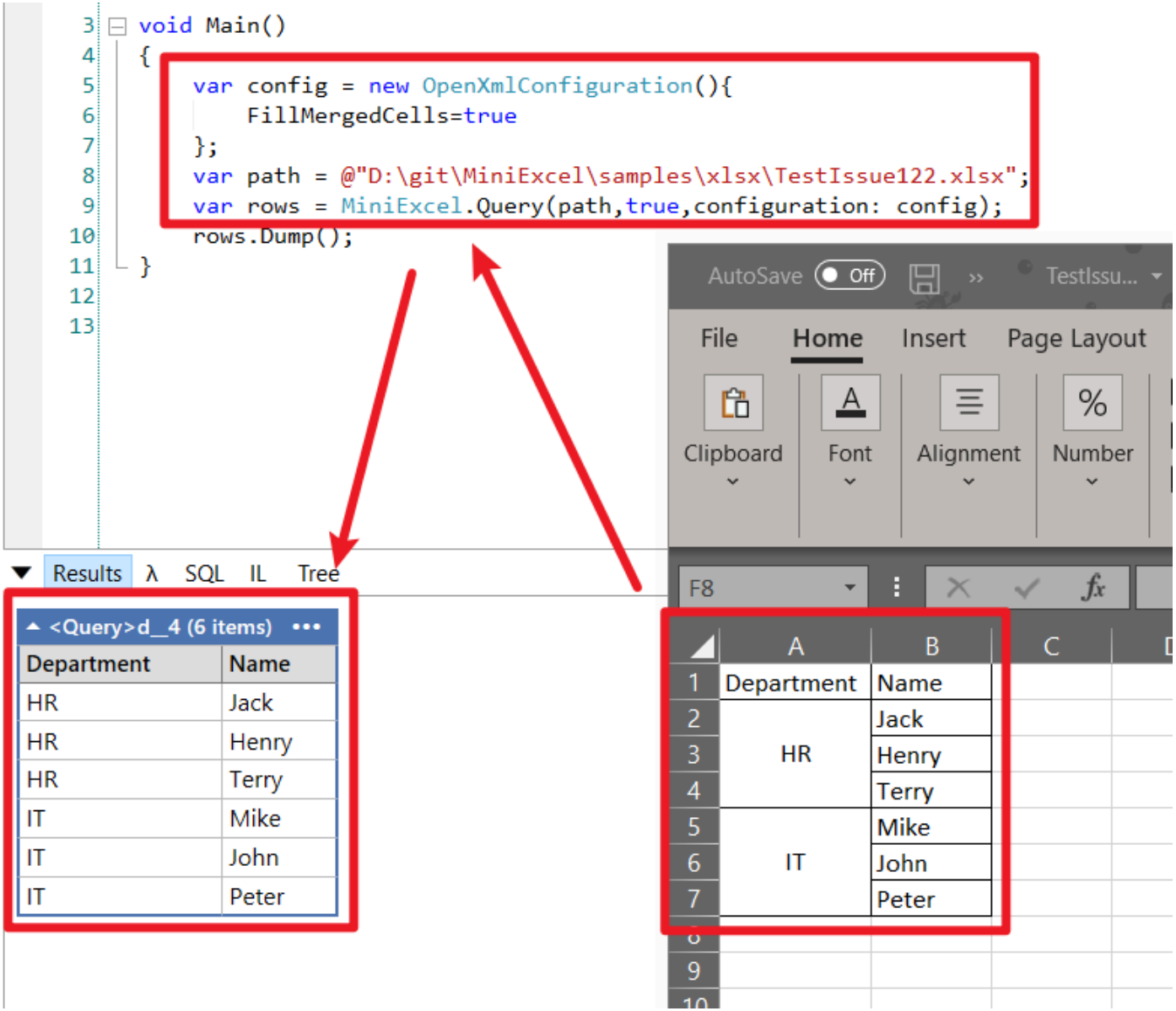


11. Fill Merged Cells

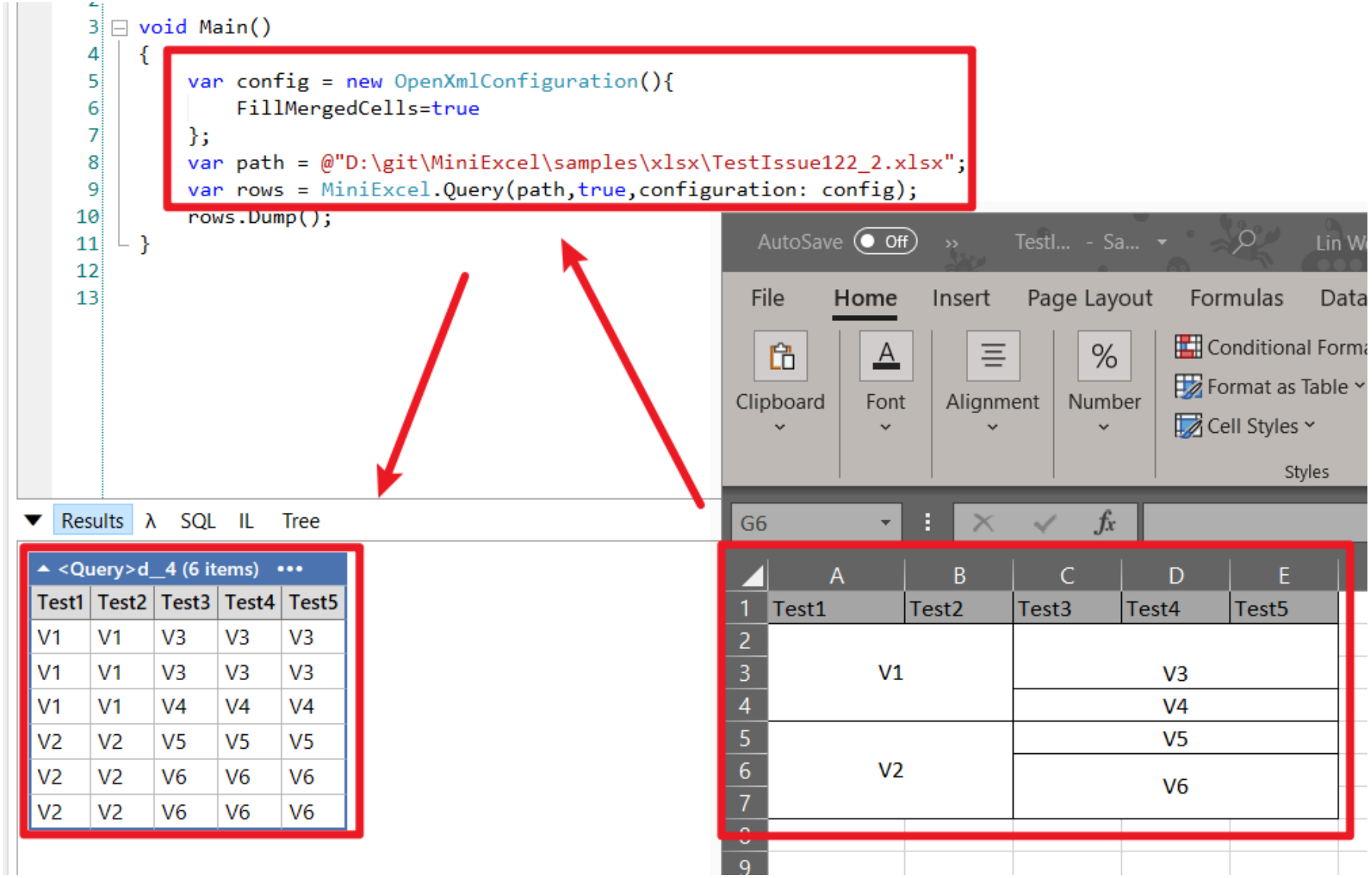
Note: The efficiency is slower compared to `not using merge fill`

Reason: The OpenXml standard puts mergeCells at the bottom of the file, which leads to the need to foreach the sheetxml twice

```
var config = new OpenXmlConfiguration()
{
    FillMergedCells = true
};
var rows = MiniExcel.Query(path, configuration: config);
```



support variable length and width multi-row and column filling



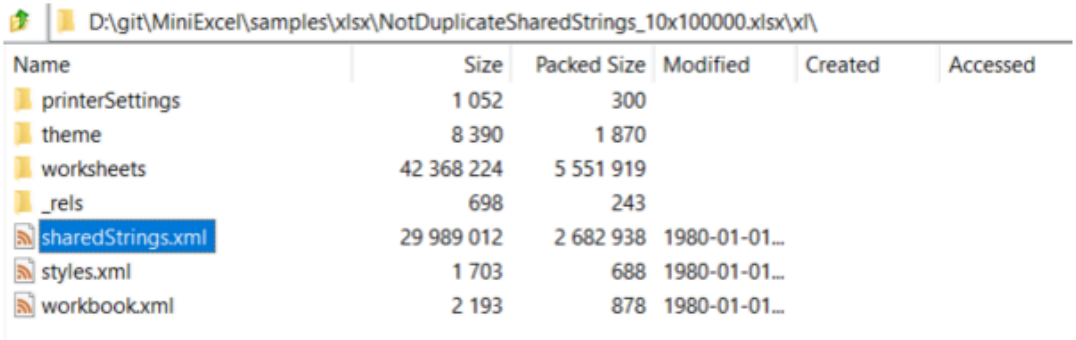
12. Reading big file by disk-base cache (Disk-Base Cache - SharedString)

If the SharedStrings size exceeds 5 MB, MiniExcel default will use local disk cache, e.g, 10x100000.xlsx(one million rows data), when disable disk cache the maximum memory usage is 195MB, but able disk cache only needs 65MB. Note, this optimization needs some efficiency cost, so this case will increase reading time from 7.4 seconds to 27.2 seconds, If you don't need it that you can disable disk cache with the following code:

```
var config = new OpenXmlConfiguration { EnableSharedStringCache = false };
MiniExcel.Query(path,configuration: config)
```

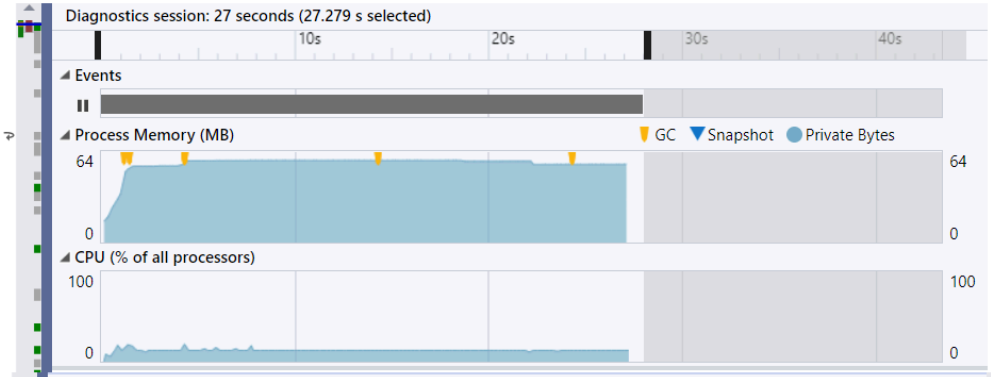
You can use `SharedStringCacheSize` to change the sharedString file size beyond the specified size for disk caching

```
var config = new OpenXmlConfiguration { SharedStringCacheSize=500*1024*1024 };
MiniExcel.Query(path, configuration: config);
```

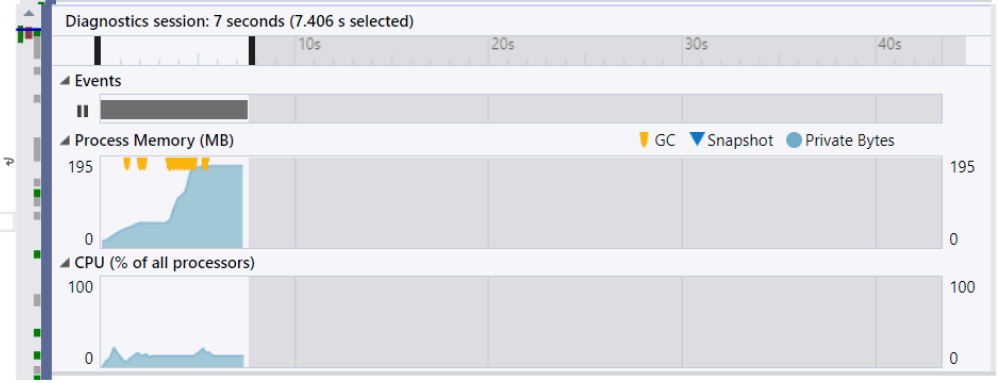


Key1	Key2	Key3	Key4	Key5	Key6	Key7	Key8	Key9	Key10
Key1Value1	Key2Value1	Key3Value1	Key4Value1	Key5Value1	Key6Value1	Key7Value1	Key8Value1	Key9Value1	Key10Value1
Key1Value2	Key2Value2	Key3Value2	Key4Value2	Key5Value2	Key6Value2	Key7Value2	Key8Value2	Key9Value2	Key10Value2
Key1Value3	Key2Value3	Key3Value3	Key4Value3	Key5Value3	Key6Value3	Key7Value3	Key8Value3	Key9Value3	Key10Value3
Key1Value4	Key2Value4	Key3Value4	Key4Value4	Key5Value4	Key6Value4	Key7Value4	Key8Value4	Key9Value4	Key10Value4
Key1Value5	Key2Value5	Key3Value5	Key4Value5	Key5Value5	Key6Value5	Key7Value5	Key8Value5	Key9Value5	Key10Value5
Key1Value6	Key2Value6	Key3Value6	Key4Value6	Key5Value6	Key6Value6	Key7Value6	Key8Value6	Key9Value6	Key10Value6
Key1Value7	Key2Value7	Key3Value7	Key4Value7	Key5Value7	Key6Value7	Key7Value7	Key8Value7	Key9Value7	Key10Value7
Key1Value8	Key2Value8	Key3Value8	Key4Value8	Key5Value8	Key6Value8	Key7Value8	Key8Value8	Key9Value8	Key10Value8
Key1Value9	Key2Value9	Key3Value9	Key4Value9	Key5Value9	Key6Value9	Key7Value9	Key8Value9	Key9Value9	Key10Value9
Key1Value10	Key2Value10	Key3Value10	Key4Value10	Key5Value10	Key6Value10	Key7Value10	Key8Value10	Key9Value10	Key10Value10
Key1Value11	Key2Value11	Key3Value11	Key4Value11	Key5Value11	Key6Value11	Key7Value11	Key8Value11	Key9Value11	Key10Value11
Key1Value12	Key2Value12	Key3Value12	Key4Value12	Key5Value12	Key6Value12	Key7Value12	Key8Value12	Key9Value12	Key10Value12
Key1Value13	Key2Value13	Key3Value13	Key4Value13	Key5Value13	Key6Value13	Key7Value13	Key8Value13	Key9Value13	Key10Value13
Key1Value14	Key2Value14	Key3Value14	Key4Value14	Key5Value14	Key6Value14	Key7Value14	Key8Value14	Key9Value14	Key10Value14
Key1Value15	Key2Value15	Key3Value15	Key4Value15	Key5Value15	Key6Value15	Key7Value15	Key8Value15	Key9Value15	Key10Value15
Key1Value16	Key2Value16	Key3Value16	Key4Value16	Key5Value16	Key6Value16	Key7Value16	Key8Value16	Key9Value16	Key10Value16
Key1Value17	Key2Value17	Key3Value17	Key4Value17	Key5Value17	Key6Value17	Key7Value17	Key8Value17	Key9Value17	Key10Value17
Key1Value18	Key2Value18	Key3Value18	Key4Value18	Key5Value18	Key6Value18	Key7Value18	Key8Value18	Key9Value18	Key10Value18
Key1Value19	Key2Value19	Key3Value19	Key4Value19	Key5Value19	Key6Value19	Key7Value19	Key8Value19	Key9Value19	Key10Value19
Key1Value20	Key2Value20	Key3Value20	Key4Value20	Key5Value20	Key6Value20	Key7Value20	Key8Value20	Key9Value20	Key10Value20
Key1Value21	Key2Value21	Key3Value21	Key4Value21	Key5Value21	Key6Value21	Key7Value21	Key8Value21	Key9Value21	Key10Value21
Key1Value22	Key2Value22	Key3Value22	Key4Value22	Key5Value22	Key6Value22	Key7Value22	Key8Value22	Key9Value22	Key10Value22
Key1Value23	Key2Value23	Key3Value23	Key4Value23	Key5Value23	Key6Value23	Key7Value23	Key8Value23	Key9Value23	Key10Value23

```
[Fact]
0 references
public void TestIssuePerformance()
{
    var path = PathHelper.GetFile("xlsx/NotDuplicateSharedStrings_10x100000.xlsx");
    foreach (var sheetName in MiniExcel.GetSheetNames(path))
        foreach (var item in MiniExcel.Query(path, useHeaderRow: true, sheetName:
          sheetName))
            Assert.Contains("Key", item.Key1);
}
```




```
[Fact]
0 references
public void TestIssuePerformance()
{
    var config = new OpenXmlConfiguration { EnableSharedStringCache = false };
    var path = PathHelper.GetFile("xlsx/NotDuplicateSharedStrings_10x100000.xlsx");
    foreach (var sheetName in MiniExcel.GetSheetNames(path))
        foreach (var item in MiniExcel.Query(path, useHeaderRow: true, sheetName:
          sheetName,configuration: config))
            Assert.Contains("Key", item.Key1);
}
```



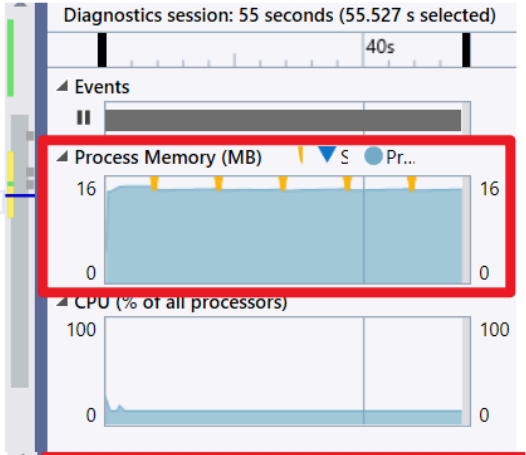
Create/Export Excel

1. Must be a non-abstract type with a public parameterless constructor .

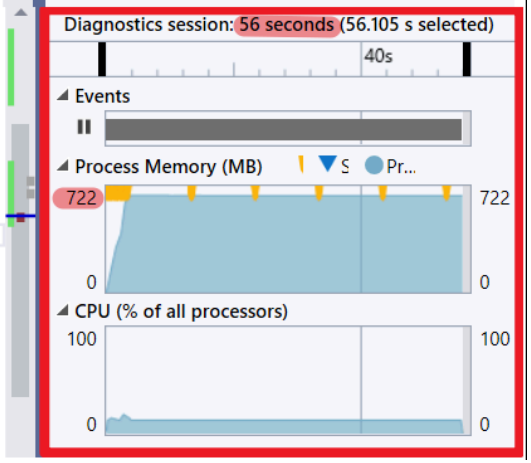
2. MiniExcel support parameter IEnumerable Deferred Execution, If you want to use least memory, please do not call methods such as ToList

e.g : ToList or not memory usage

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        var values = Enumerable.Range(1, 10000000).Select((s, index) => new { index, value = Guid.NewGuid() });
        var path = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid().ToString()}.xlsx");
        using (var stream = File.Create(path))
            stream.SaveAs(values);
    }
}
```




```
class Program
{
    0 references
    static void Main(string[] args)
    {
        var values = Enumerable.Range(1, 10000000).Select((s, index) => new { index, value = Guid.NewGuid() }).ToList();
        var path = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid().ToString()}.xlsx");
        using (var stream = File.Create(path))
            stream.SaveAs(values);
    }
}
```



1. Anonymous or strongly type [Try it]

```
var path = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid()}.xlsx");
MiniExcel.SaveAs(path, new[] {
    new { Column1 = "MiniExcel", Column2 = 1 },
    new { Column1 = "Github", Column2 = 2}
});
```

2. IEnumerable<IDictionary<string, object>>

```
var values = new List<Dictionary<string, object>>()
{
    new Dictionary<string,object>{{ "Column1", "MiniExcel" }, { "Column2", 1 } },
    new Dictionary<string,object>{{ "Column1", "Github" }, { "Column2", 2 } }
};
MiniExcel.SaveAs(path, values);
```

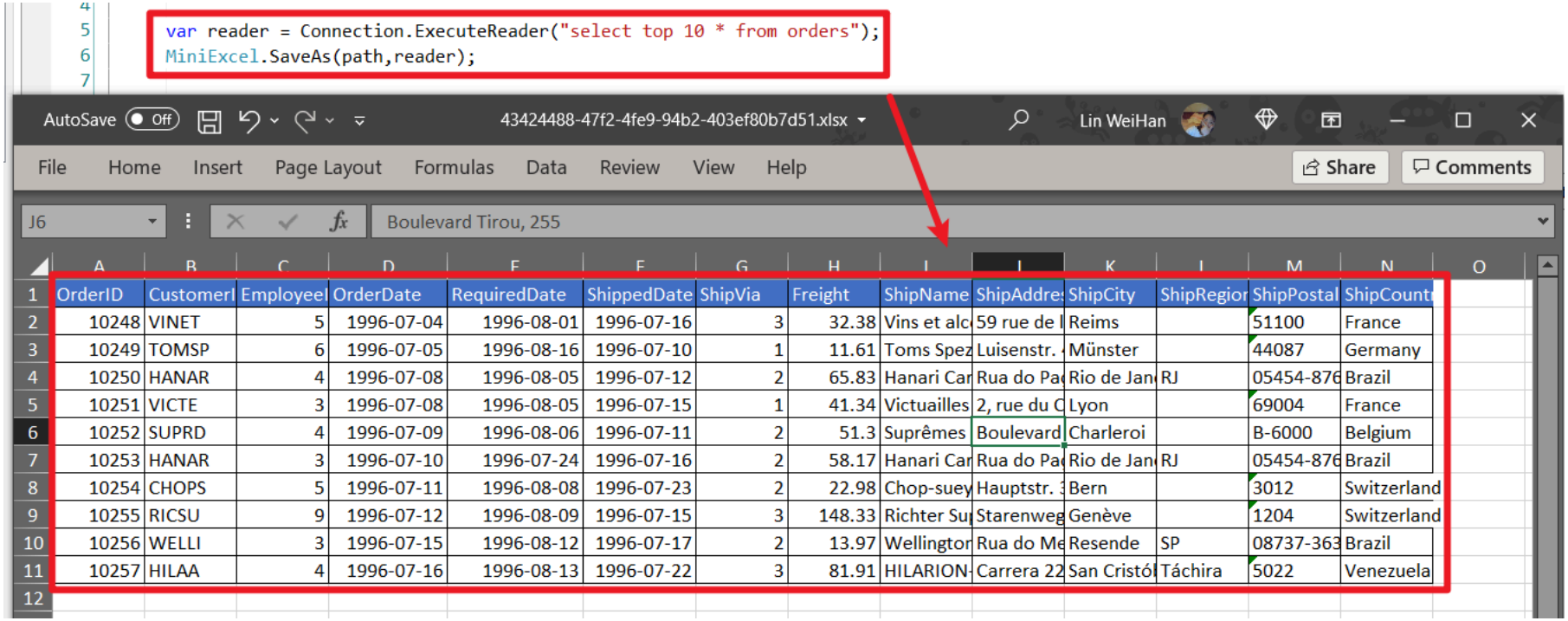
Create File Result :

Column1	Column2
MiniExcel	1
Github	2

3. IDataReader

- `Recommended` , it can avoid to load all data into memory

```
MiniExcel.SaveAs(path, reader);
```



DataReader export multiple sheets (recommand by Dapper ExecuteReader)

```
using (var cnn = Connection)
{
    cnn.Open();
    var sheets = new Dictionary<string,object>();
    sheets.Add("sheet1", cnn.ExecuteReader("select 1 id"));
    sheets.Add("sheet2", cnn.ExecuteReader("select 2 id"));
    MiniExcel.SaveAs("Demo.xlsx", sheets);
}
```

4. Datatable

- `Not recommended` , it will load all data into memory
- DataTable use Caption for column name first, then use columname

```
var path = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid()}.xlsx");
var table = new DataTable();
{
    table.Columns.Add("Column1", typeof(string));
    table.Columns.Add("Column2", typeof(decimal));
    table.Rows.Add("MiniExcel", 1);
    table.Rows.Add("Github", 2);
}

MiniExcel.SaveAs(path, table);
```

5. Dapper Query

Thanks @shaofing #552 , please use `CommandDefinition + CommandFlags.NoCache`

```
using (var connection = GetConnection(connectionString))
{
    var rows = connection.Query(
        new CommandDefinition(
            @"select 'MiniExcel' as Column1,1 as Column2 union all select 'Github',2"
            , flags: CommandFlags.NoCache)
        );
    // Note: QueryAsync will throw close connection exception
    MiniExcel.SaveAs(path, rows);
}
```

Below code will load all data into memory

```
using (var connection = GetConnection(connectionString))
{
    var rows = connection.Query(@"select 'MiniExcel' as Column1,1 as Column2 union all select 'Github',2");
    MiniExcel.SaveAs(path, rows);
}
```

6. SaveAs to MemoryStream [Try it]

```
using (var stream = new MemoryStream()) //support FileStream,MemoryStream ect.
{
    stream.SaveAs(values);
}
```

e.g : api of export excel

```
public IActionResult DownloadExcel()
{
    var values = new[] {
        new { Column1 = "MiniExcel", Column2 = 1 },
        new { Column1 = "Github", Column2 = 2}
    };

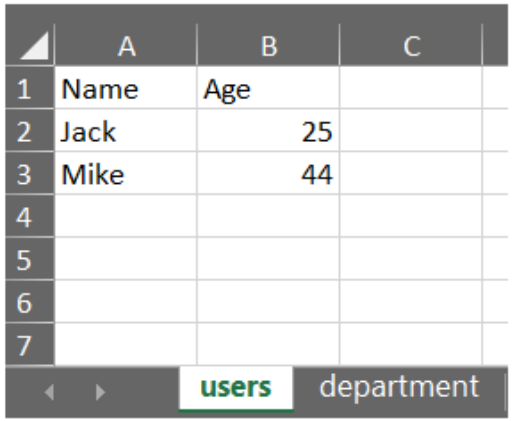
    var memoryStream = new MemoryStream();
    memoryStream.SaveAs(values);
    memoryStream.Seek(0, SeekOrigin.Begin);
    return new FileStreamResult(memoryStream, "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
    {
        FileDownloadName = "demo.xlsx"
    };
}
```

7. Create Multiple Sheets

```
// 1. Dictionary<string,object>
var users = new[] { new { Name = "Jack", Age = 25 }, new { Name = "Mike", Age = 44 } };
var department = new[] { new { ID = "01", Name = "HR" }, new { ID = "02", Name = "IT" } };
var sheets = new Dictionary<string, object>
{
    ["users"] = users,
    ["department"] = department
};
MiniExcel.SaveAs(path, sheets);

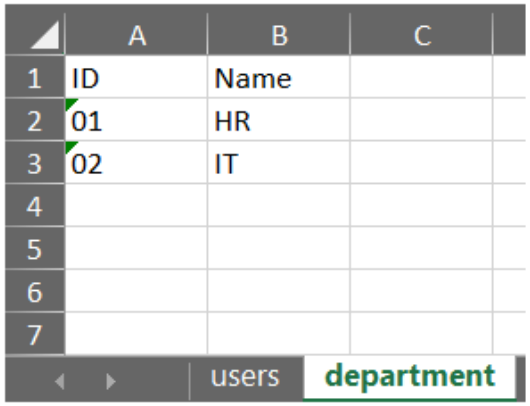
// 2. DataSet
var sheets = new DataSet();
sheets.Add(UsersDataTable);
```

```
sheets.Add(DepartmentDataTable);
//..
MiniExcel.SaveAs(path, sheets);
```



	A	B	C
1	Name	Age	
2	Jack	25	
3	Mike	44	
4			
5			
6			
7			

users | department

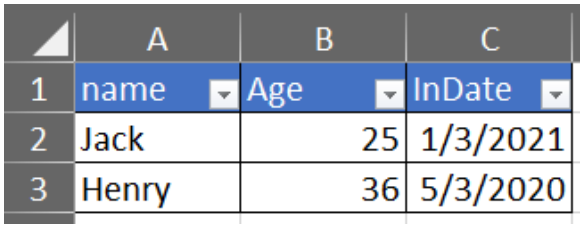


	A	B	C
1	ID	Name	
2	01	HR	
3	02	IT	
4			
5			
6			
7			

users | department

8. TableStyles Options

Default style



	A	B	C
1	name	Age	InDate
2	Jack	25	1/3/2021
3	Henry	36	5/3/2020

Without style configuration

```
var config = new OpenXmlConfiguration()
{
     TableStyles = TableStyles.None
};
MiniExcel.SaveAs(path, value,configuration:config);
```

	A	B	C
1	name	Age	InDate
2	Jack	25	2021-01-03
3	Henry	36	2020-05-03
4			

9. AutoFilter

Since v0.19.0 `OpenXmlConfiguration.AutoFilter` can en/unable AutoFilter , default value is `true` , and setting AutoFilter way:

```
MiniExcel.SaveAs(path, value, configuration: new OpenXmlConfiguration() { AutoFilter = false });
```

10. Create Image

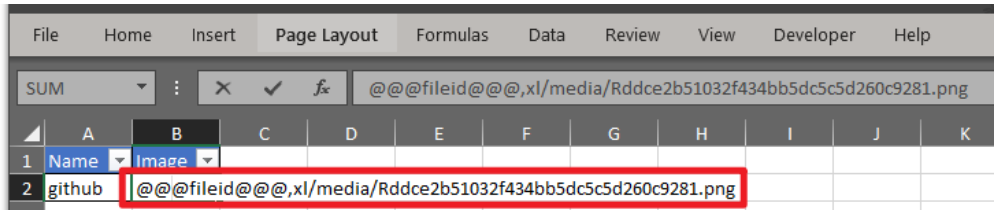
```
var value = new[] {
    new { Name="github",Image=File.ReadAllBytes(PathHelper.GetFile("images/github_logo.png"))},
    new { Name="google",Image=File.ReadAllBytes(PathHelper.GetFile("images/google_logo.png"))},
    new { Name="microsoft",Image=File.ReadAllBytes(PathHelper.GetFile("images/microsoft_logo.png"))},
    new { Name="reddit",Image=File.ReadAllBytes(PathHelper.GetFile("images/reddit_logo.png"))},
    new { Name="statck_overflow",Image=File.ReadAllBytes(PathHelper.GetFile("images/statck_overflow_logo.png"))},
};
MiniExcel.SaveAs(path, value);
```

	A	B
1	Name	Image
2	github	
3	google	
4	microsoft	
5	reddit	
6	statck_overflow	

11. Byte Array File Export

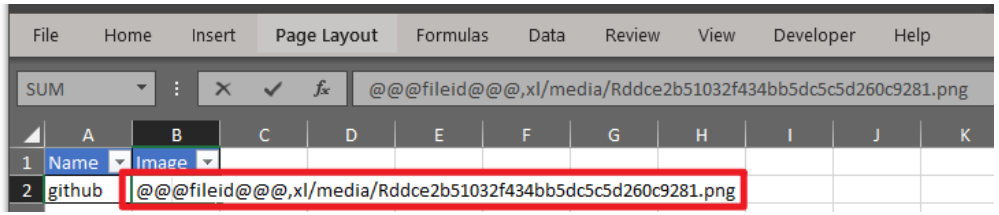
Since 1.22.0, when value type is `byte[]` then system will save file path at cell by default, and when import system can be converted to `byte[]` . And if you don't want to use it, you can set `OpenXmlConfiguration.EnableConvertByteArray` to `false` , it can improve the system efficiency.

```
public void TestIssue318()
{
    var imageByte = File.ReadAllBytes(PathHelper.GetFile("images/github_logo.png"));
    var path = PathHelper.GetTempFilePath();
    var value = new[] {
        new { Name="github",Image=imageByte},
    };
    MiniExcel.SaveAs(path, value);
```



Since 1.22.0, when value type is `byte[]` then system will save file path at cell by default, and when import system can be converted to `byte[]`. And if you don't want to use it, you can set `OpenXmlConfiguration.EnableConvertByteArray` to `false`, it can improve the system efficiency.

```
public void TestIssue318()
{
    var imageByte = File.ReadAllBytes(PathHelper.GetFile("images/github_logo.png"));
    var path = PathHelper.GetTempFilePath();
    var value = new[] {
        new { Name="github",Image=imageByte},
    };
    MiniExcel.SaveAs(path, value);
```



12. Merge same cells vertically

This functionality is only supported in `xlsx` format and merges cells vertically between @merge and @endmerge tags. You can use @mergelimit to limit boundaries of merging cells vertically.

```
var mergedFilePath = Path.Combine(Path.GetTempPath(), ${Guid.NewGuid().ToString()}.xlsx");

var path = @"../../../../../samples/xlsx/TestMergeWithTag.xlsx";

MiniExcel.MergeSameCells(mergedFilePath, path);
```

```
var memoryStream = new MemoryStream();

var path = @"../../../../../samples/xlsx/TestMergeWithTag.xlsx";

memoryStream.MergeSameCells(path);
```

File content before and after merge:

Without merge limit:

	A	B	C	D
1	Department	Name	Name	Date
2	@merge		@merge	
3	HR	Jack	Jack	05.02.2023
4	HR	Henry	Henry	05.02.2023
5	HR	Peter	Henry	13.02.2023
6	@endmerge		@endmerge	
7	Henry	Mike	Mike	15.02.2023
8	Henry	Mike	Mike	15.02.2023
9	@merge		@merge	
10	IT	Mike	Mike	15.02.2023
11	IT	Peter	Peter	15.02.2023
12	@endmerge		@endmerge	
13				

	A	B	C	D
1	Department	Name	Name	Date
2	HR	Jack	Jack	05.02.2023
3		Henry	Henry	05.02.2023
4		Peter		13.02.2023
5	Henry	Mike	Mike	15.02.2023
6	Henry	Mike	Mike	15.02.2023
7		Mike	Mike	15.02.2023
8	IT	Peter	Peter	15.02.2023
9				
10				
11				
12				
13				

With merge limit:

	A	B	C	D
1	Department	Name	Name	Date
2	@merge		@mergelimit	
3	HR	Jack	Jack	05.02.2023
4	HR	Henry	Henry	05.02.2023
5	HR	Peter	Henry	13.02.2023
6	Henry	Mike	Henry	15.02.2023
7	Henry	Mike	Henry	15.02.2023
8	@endmerge		@endmerge	
9	IT	Mike	Mike	15.02.2023
10	IT	Peter	Peter	15.02.2023
11				
12				

	A	B	C	D
1	Department	Name	Name	Date
2	HR	Jack	Jack	05.02.2023
3	HR	Henry	Henry	05.02.2023
4		Peter		13.02.2023
5	Henry	Mike	Henry	15.02.2023
6		Mike		15.02.2023
7	IT	Mike	Mike	15.02.2023
8	IT	Peter	Peter	15.02.2023
9				
10				
11				
12				

13. Skip null values

New explicit option to write empty cells for null values:

```
DataTable dt = new DataTable();

/* ... */

DataRow dr = dt.NewRow();

dr["Name1"] = "Somebody once";
dr["Name2"] = null;
dr["Name3"] = "told me.";
```

```
dt.Rows.Add(dr);

OpenXmlConfiguration configuration = new OpenXmlConfiguration()
{
     EnableWriteNullValueCell = true // Default value.
};

MiniExcel.SaveAs(@"C:\temp\Book1.xlsx", dt, configuration: configuration);
```

Name1	Name2	Name3
Somebody once		told me.

```
<x:row r="2">
    <x:c r="A2" t ="str" s="2">
        <x:v>Somebody once</x:v>
    </x:c>
    <x:c r="B2" s="2"></x:c>
    <x:c r="C2" t ="str" s="2">
        <x:v>told me.</x:v>
    </x:c>
</x:row>
```

Previous behavior:

```
/* ... */

OpenXmlConfiguration configuration = new OpenXmlConfiguration()
{
     EnableWriteNullValueCell = false // Default value is true.
};

MiniExcel.SaveAs(@"C:\temp\Book1.xlsx", dt, configuration: configuration);
```

Name1	Name2	Name3
Somebod		told me.

```
<x:row r="2">
    <x:c r="A2" t ="str" s="2">
        <x:v>Somebody once</x:v>
    </x:c>
    <x:c r="B2" t ="str" s="2">
        <x:v></x:v>
    </x:c>
    <x:c r="C2" t ="str" s="2">
        <x:v>told me.</x:v>
    </x:c>
</x:row>
```

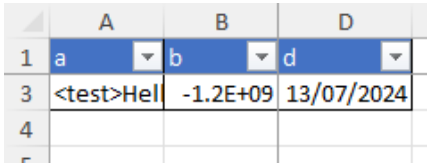
Works for null and DBNull values.

14. Freeze Panes

```
/* ... */

OpenXmlConfiguration configuration = new OpenXmlConfiguration()
{
    FreezeRowCount = 1,     // default is 1
    FreezeColumnCount = 2   // default is 0
};

MiniExcel.SaveAs(@"C:\temp\Book1.xlsx", dt, configuration: configuration);
```



	A	B	D
1	a	b	d
3	<test>Hel	-1.2E+09	13/07/2024
4			

Fill Data To Excel Template

- The declaration is similar to Vue template `{{variable name}}` , or the collection rendering `{{collection name.field name}}`
- Collection rendering support IEnumerable/DataTable/DapperRow

1. Basic Fill

Template:

	A	B	C	D	E
1	Name	CreateDate	VIP	Points	REM
2	{{Name}}	{{CreateDate}}	{{VIP}}	{{Points}}	{{Name}} has {{Points}} points

Result:

	A	B	C	D	E
1	Name	CreateDate	VIP	Points	REM
2	Jack	2021-01-01 12:00:00	TRUE	123	Jack has 123 points

Code:

```
// 1. By POCO
var value = new
{
    Name = "Jack",
    CreateDate = new DateTime(2021, 01, 01),
    VIP = true,
    Points = 123
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);


// 2. By Dictionary
var value = new Dictionary<string, object>()
{
    ["Name"] = "Jack",
    ["CreateDate"] = new DateTime(2021, 01, 01),
    ["VIP"] = true,
    ["Points"] = 123
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);
```

2. IEnumerable Data Fill

> Note1: Use the first IEnumerable of the same column as the basis for filling list

Template:

	A	B
1	Name	Department
2	{{employees.name}}	{{employees.department}}

Result:

	A	B
1	Name	Department
2	Jack	HR
3	Lisa	HR
4	John	HR
5	Mike	IT
6	Neo	IT
7	Loan	IT

Code:

```
//1. By POCO
var value = new
{
    employees = new[] {
        new {name="Jack",department="HR"},
        new {name="Lisa",department="HR"},
        new {name="John",department="HR"},
        new {name="Mike",department="IT"},
        new {name="Neo",department="IT"},
        new {name="Loan",department="IT"}
    }
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);

//2. By Dictionary
var value = new Dictionary<string, object>()
{
    ["employees"] = new[] {
        new {name="Jack",department="HR"},
        new {name="Lisa",department="HR"},
        new {name="John",department="HR"},
        new {name="Mike",department="IT"},
        new {name="Neo",department="IT"},
        new {name="Loan",department="IT"}
    }
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);
```

3. Complex Data Fill

> Note: Support multi-sheets and using same varible

Template:

	A	B	C
1	{{title}}		
2	Managers	name	department
3		{{managers.name}}	{{managers.department}}
4	Employees	name	department
5		{{employees.name}}	{{employees.department}}

Result:

	A	B	C
1	FooCompany		
2	Managers	name	department
3		Jack	HR
4		Loan	IT
5	Employees	name	department
6		Wade	HR
7		Felix	HR
8		Eric	IT
9		Keaton	IT

```
// 1. By POCO
var value = new
{
    title = "FooCompany",
    managers = new[] {
        new {name="Jack",department="HR"},
        new {name="Loan",department="IT"}
    },
    employees = new[] {
        new {name="Wade",department="HR"},
```

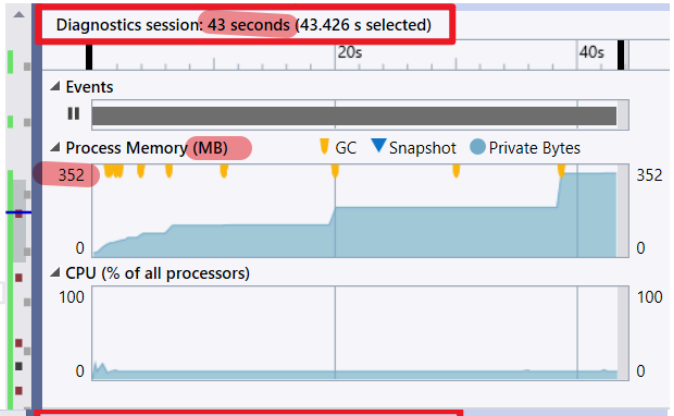
```
        new {name="Felix",department="HR"},
        new {name="Eric",department="IT"},
        new {name="Keaton",department="IT"}
    }
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);

// 2. By Dictionary
var value = new Dictionary<string, object>()
{
    ["title"] = "FooCompany",
    ["managers"] = new[] {
        new {name="Jack",department="HR"},
        new {name="Loan",department="IT"}
    },
    ["employees"] = new[] {
        new {name="Wade",department="HR"},
        new {name="Felix",department="HR"},
        new {name="Eric",department="IT"},
        new {name="Keaton",department="IT"}
    }
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);
```

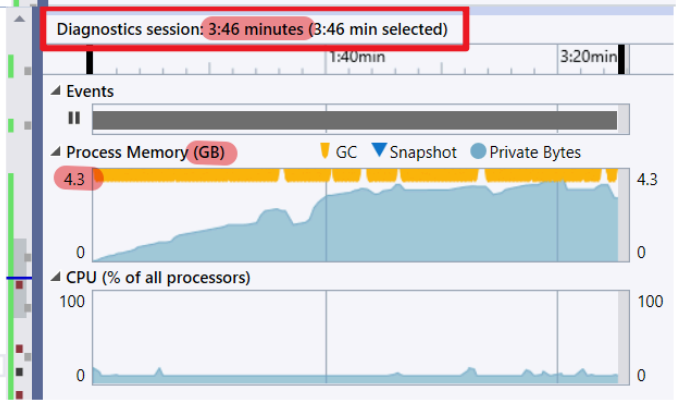
4. Fill Big Data Performance

> NOTE: Using IEnumerable deferred execution not ToList can save max memory usage in MiniExcel

```cs
// MiniExcel
{
    var path = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid().ToString()}.xlsx");
    var templatePath = @"..\..\..\..\..\samples\xlsx\TestTemplateBasicIEmumerableFill.xlsx";
    var value = new
    {
        employees = Enumerable.Range(1, 1000000).Select(s => new { name = "Jack", department = "HR" })
    };
    MiniExcel.SaveAsByTemplate(path, templatePath, value);
}
```




```cs
// ClosexXml.Report
{
    var path = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid().ToString()}.xlsx");
    var templatePath = @"..\..\..\..\..\samples\xlsx\TestTemplateBasicIEmumerableFill_ClosedXML_Report.xlsx";
    var template = new ClosedXML.Report.XLTemplate(templatePath);
    var value = new
    {
        employees = Enumerable.Range(1, 1000000).Select(s => new { name = "Jack", department = "HR" })
    };
    template.AddVariable(value);
    template.Generate();
    template.SaveAs(path);
```



5. Cell value auto mapping type

Template

	A	B	C	D	E	F	G
1	string	int	decimal	double	datetime	bool	Guid
2	{{Ts.string}}	{{Ts.int}}	{{Ts.decimal}}	{{Ts.double}}	{{Ts.datetime}}	{{Ts.bool}}	{{Ts.Guid}}

Result

	A	B	C	D	E	F	G	H	I	J
1	string	int	decimal	double	datetime	bool	Guid			
2	string	123	123.45	123.33	2021-04-01 00:0	TRUE	0387ef65-9e30-4be7-85b5-e42a947b06d7			
3										
4										
5	string	123	123.45	123.33	2021-04-01 00:0	TRUE	0387ef65-9e30-4be7-85b5-e42a947b06d7			

Class

```
public class Poco
{
    public string @string { get; set; }
    public int? @int { get; set; }
    public decimal? @decimal { get; set; }
    public double? @double { get; set; }
    public DateTime? datetime { get; set; }
    public bool? @bool { get; set; }
    public Guid? Guid { get; set; }
}
```

Code

```
var poco = new TestIEnumerableTypePoco { @string = "string", @int = 123, @decimal = decimal.Parse("123.45"), @double = (double)
var value = new
{
    Ts = new[] {
        poco,
        new TestIEnumerableTypePoco{},
        null,
        poco
    }
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);
```

6. Example : List Github Projects

Template

	A	B	C	D
1	GitHub			
2		{{User}} Github Projects		
3				
4				
5	Name	Link	Star	CreateTime
6	{{Projects.Name}}	{{Projects.Link}}	{{Projects.Star}}	{{Projects.CreateTime}}
7			Total Star : {{TotalStar}}	

Result

	A	B	C	D
1	GitHub			
2		ITWeiHan Github Projects		
3				
4				
5	Name	Link	Star	CreateTime
6	MiniExcel	https://github.com/shp	146	2021-03-01 00:00:00
7	HtmlTableHelper	https://github.com/shp	16	2020-02-01 00:00:00
8	PocoClassGenerator	https://github.com/shp	16	2019-03-17 00:00:00
9			Total Star : 178	

Code

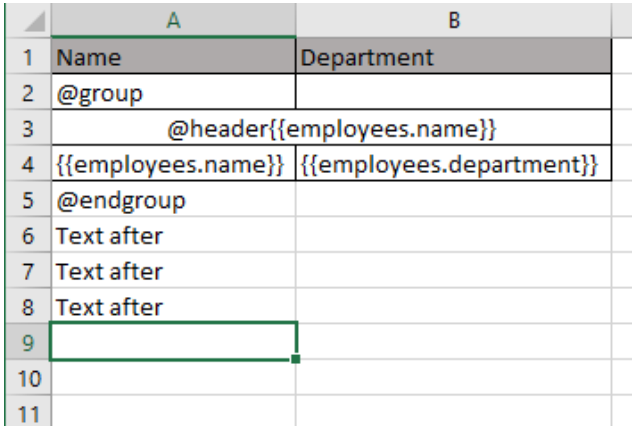
```
var projects = new[]
{
    new {Name = "MiniExcel",Link="https://github.com/mini-software/MiniExcel",Star=146, CreateTime=new DateTime(2021,03,01)},
    new {Name = "HtmlTableHelper",Link="https://github.com/mini-software/HtmlTableHelper",Star=16, CreateTime=new DateTime(2020
    new {Name = "PocoClassGenerator",Link="https://github.com/mini-software/PocoClassGenerator",Star=16, CreateTime=new DateTim
};
var value = new
{
    User = "ITWeiHan",
    Projects = projects,
    TotalStar = projects.Sum(s => s.Star)
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);
```

7. Grouped Data Fill

```
var value = new Dictionary<string, object>()
{
    ["employees"] = new[] {
        new {name="Jack",department="HR"},
        new {name="Jack",department="HR"},
        new {name="John",department="HR"},
        new {name="John",department="IT"},
        new {name="Neo",department="IT"},
        new {name="Loan",department="IT"}
    }
};
await MiniExcel.SaveAsByTemplateAsync(path, templatePath, value);
```

1. With `@group` tag and with `@header` tag

Before



	A	B
1	Name	Department
2	@group	
3	@header{{employees.name}}	
4	{{employees.name}}	{{employees.department}}
5	@endgroup	
6	Text after	
7	Text after	
8	Text after	
9		
10		
11		

After

	A	B
1	Name	Department
2		Jack
3	Jack	HR
4	Jack	HR
5		John
6	John	HR
7	John	IT
8		Neo
9	Neo	IT
10		Loan
11	Loan	IT
12	Text after	
13	Text after	
14	Text after	

2. With @group tag and without @header tag

Before

	A	B
1	Name	Department
2	@group	
3	{{employees.name}}	
4	{{employees.name}}	{{employees.department}}
5	@endgroup	
6	Text after	
7	Text after	
8	Text after	
9		

After

	A	B
1	Name	Department
2		Jack
3	Jack	HR
4		Jack
5	Jack	HR
6		John
7	John	HR
8		John
9	John	IT
10		Neo
11	Neo	IT
12		Loan
13	Loan	IT
14	Text after	
15	Text after	
16	Text after	

3. Without @group tag

Before

	A	B
1	Name	Department
2	{{employees.name}}	
3	{{employees.name}}	{{employees.department}}
4	Text after	
5	Text after	
6	Text after	

After

	A	B
1	Name	Department
2		Jack
3		Jack
4		John
5		John
6		Neo
7		Loan
8	Jack	HR
9	Jack	HR
10	John	HR
11	John	IT
12	Neo	IT
13	Loan	IT
14	Text after	
15	Text after	
16	Text after	

8. If/ElseIf/Else Statements inside cell

Rules:

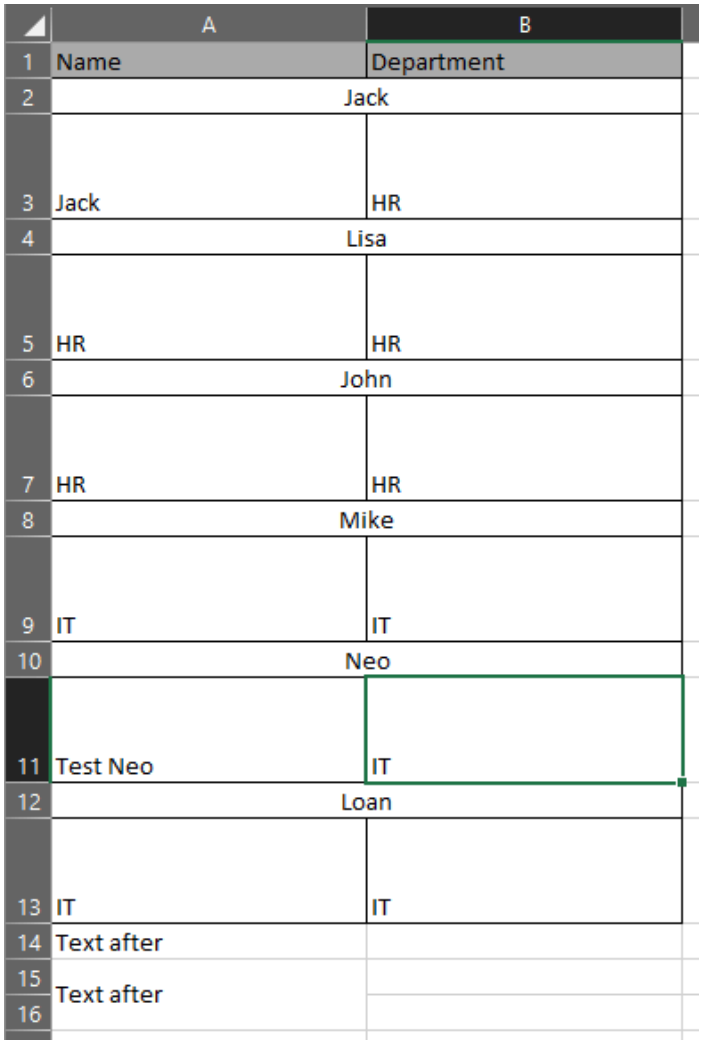
1. Supports DateTime, Double, Int with ==, !=, >, >=, <, <= operators.
2. Supports String with ==, != operators.
3. Each statement should be new line.
4. Single space should be added before and after operators.
5. There shouldn't be new line inside of statements.
6. Cell should be in exact format as below.

```
@if(name == Jack)
{{employees.name}}
@elseif(name == Neo)
Test {{employees.name}}
@else
{{employees.department}}
@endif
```

Before

	A	B
1	Name	Department
2	@group	
3	@header{{employees.name}}	
4	@if(name == Jack) {{employees.name}} @elseif(name == Neo) Test {{employees.name}} @else {{employees.department}} @endif	{{employees.department}}
5	@endgroup	
6	Text after	
7	Text after	
8		

After



	A	B
1	Name	Department
2	Jack	
3	Jack	HR
4	Lisa	
5	HR	HR
6	John	
7	HR	HR
8	Mike	
9	IT	IT
10	Neo	
11	Test Neo	IT
12	Loan	
13	IT	IT
14	Text after	
15	Text after	
16		

9. DataTable as parameter

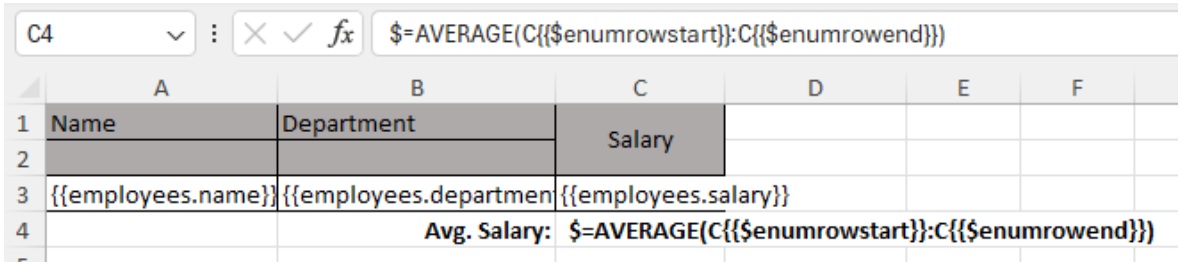
```
var managers = new DataTable();
{
    managers.Columns.Add("name");
    managers.Columns.Add("department");
    managers.Rows.Add("Jack", "HR");
    managers.Rows.Add("Loan", "IT");
}
var value = new Dictionary<string, object>()
```

```
{
    ["title"] = "FooCompany",
    ["managers"] = managers,
};
MiniExcel.SaveAsByTemplate(path, templatePath, value);
```

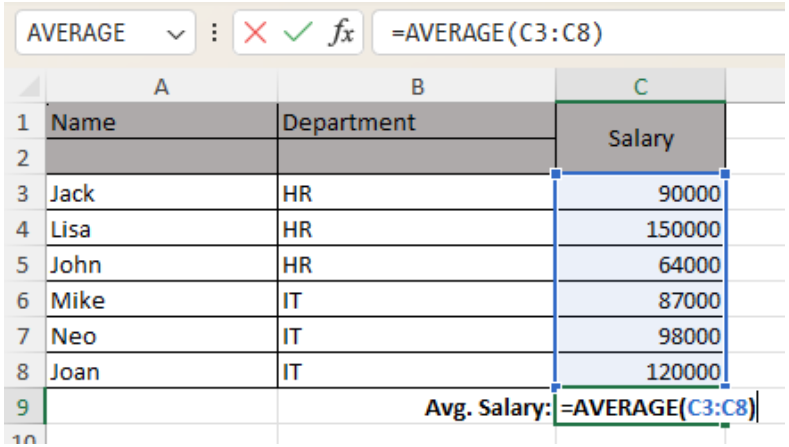
10. Formulas

1. Example

Prefix your formula with `$` and use `$enumrowstart` and `$enumrowend` to mark references to the enumerable start and end rows:



When the template is rendered, the `$` prefix will be removed and `$enumrowstart` and `$enumrowend` will be replaced with the start and end row numbers of the enumerable:



2. Other Example Formulas:

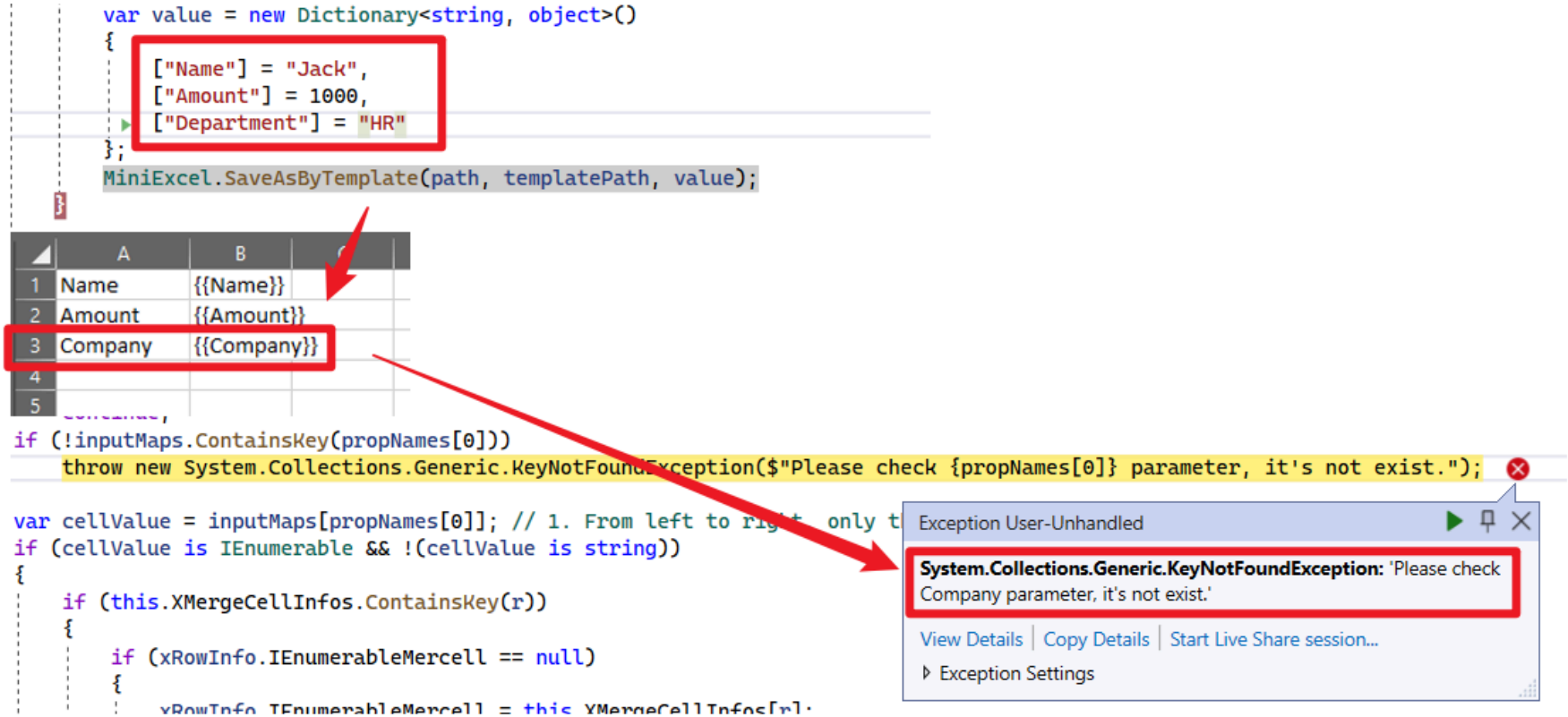
Sum	`$=SUM(C{{$enumrowstart}}:C{{$enumrowend}})`
Alt. Average	`$=SUM(C{{$enumrowstart}}:C{{$enumrowend}}) / COUNT(C{{$enumrowstart}}:C{{$enumrowend}})`
Range	`$=MAX(C{{$enumrowstart}}:C{{$enumrowend}}) - MIN(C{{$enumrowstart}}:C{{$enumrowend}})`

11. Other

1. Checking template parameter key

Since V1.24.0 , default ignore template missing parameter key and replace it with empty string, `IgnoreTemplateParameterMissing` can control throwing exception or not.

```
var config = new OpenXmlConfiguration()
{
    IgnoreTemplateParameterMissing = false,
};
MiniExcel.SaveAsByTemplate(path, templatePath, value, config)
```



Excel Column Name/Index/Ignore Attribute

1. Specify the column name, column index, column ignore

Excel Example

	A	B	C	D	E	F	G	H	I
1	**Test1**	**Test2**	**Test3**	**Test4**	**Test5**	**Test6**	**Column1**	**Column2**	**Test7**
2	Test1	Test2	Test3	Test4	Test5	Test6	Column1	Column2	Test7

Code

```
public class ExcelAttributeDemo
{
    [ExcelColumnName("Column1")]
```

```
    public string Test1 { get; set; }
    [ExcelColumnName("Column2")]
    public string Test2 { get; set; }
    [ExcelIgnore]
    public string Test3 { get; set; }
    [ExcelColumnIndex("I")] // system will convert "I" to 8 index
    public string Test4 { get; set; }
    public string Test5 { get; } //wihout set will ignore
    public string Test6 { get; private set; } //un-public set will ignore
    [ExcelColumnIndex(3)] // start with 0
    public string Test7 { get; set; }
}

var rows = MiniExcel.Query<ExcelAttributeDemo>(path).ToList();
Assert.Equal("Column1", rows[0].Test1);
Assert.Equal("Column2", rows[0].Test2);
Assert.Null(rows[0].Test3);
Assert.Equal("Test7", rows[0].Test4);
Assert.Null(rows[0].Test5);
Assert.Null(rows[0].Test6);
Assert.Equal("Test4", rows[0].Test7);
```

2. Custom Format (ExcelFormatAttribute)

Since V0.21.0 support class which contains `ToString(string content)` method format

Class

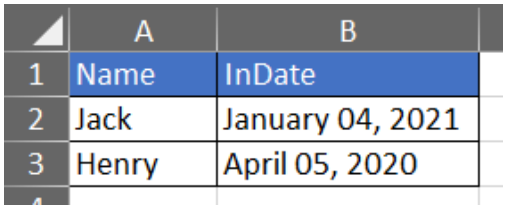
```
public class Dto
{
    public string Name { get; set; }

    [ExcelFormat("MMMM dd, yyyy")]
    public DateTime InDate { get; set; }
}
```

Code

```
var value = new Dto[] {
    new Issue241Dto{ Name="Jack",InDate=new DateTime(2021,01,04)},
    new Issue241Dto{ Name="Henry",InDate=new DateTime(2020,04,05)},
};
MiniExcel.SaveAs(path, value);
```

Result



	A	B
1	Name	InDate
2	Jack	January 04, 2021
3	Henry	April 05, 2020

Query supports custom format conversion

```
21          Console.WriteLine("==== dynamic query ====");
22          MiniExcel.Query(path, true).Dump();
23          Console.WriteLine("==== type mapping query ====");
24          MiniExcel.Query<Issue241Dto>(path).Dump();
25      }
26
27      public class Issue241Dto
28      {
29          public string Name { get; set; }
30
31          [ExcelFormat("MMMM dd, yyyy")]
32          public DateTime InDate { get; set; }
33      }
34
```

Results λ SQL IL Tree

==== dynamic query ====

<Query>d__4 (2 items)

Name	InDate
Jack	January 04, 2021
Henry	April 05, 2020

==== type mapping query ====

ø (2 items)

Name	InDate
Jack	2021-01-04 12:00:00 AM
Henry	2020-04-05 12:00:00 AM

3. Set Column Width(ExcelColumnWidthAttribute)

```
public class Dto
{
    [ExcelColumnWidth(20)]
    public int ID { get; set; }
    [ExcelColumnWidth(15.50)]
```

```
    public string Name { get; set; }
}
```

4. Multiple column names mapping to the same property.

```
public class Dto
{
    [ExcelColumnName(excelColumnName:"EmployeeNo",aliases:new[] { "EmpNo","No" })]
    public string Empno { get; set; }
    public string Name { get; set; }
}
```

5. System.ComponentModel.DisplayNameAttribute = ExcelColumnName.excelColumnNameAttribute

Since 1.24.0, system supports System.ComponentModel.DisplayNameAttribute = ExcelColumnName.excelColumnNameAttribute

```
public class TestIssueI4TXGTDto
{
    public int ID { get; set; }
    public string Name { get; set; }
    [DisplayName("Specification")]
    public string Spc { get; set; }
    [DisplayName("Unit Price")]
    public decimal Up { get; set; }
}
```

6. ExcelColumnAttribute

Since V1.26.0, multiple attributes can be simplified like :

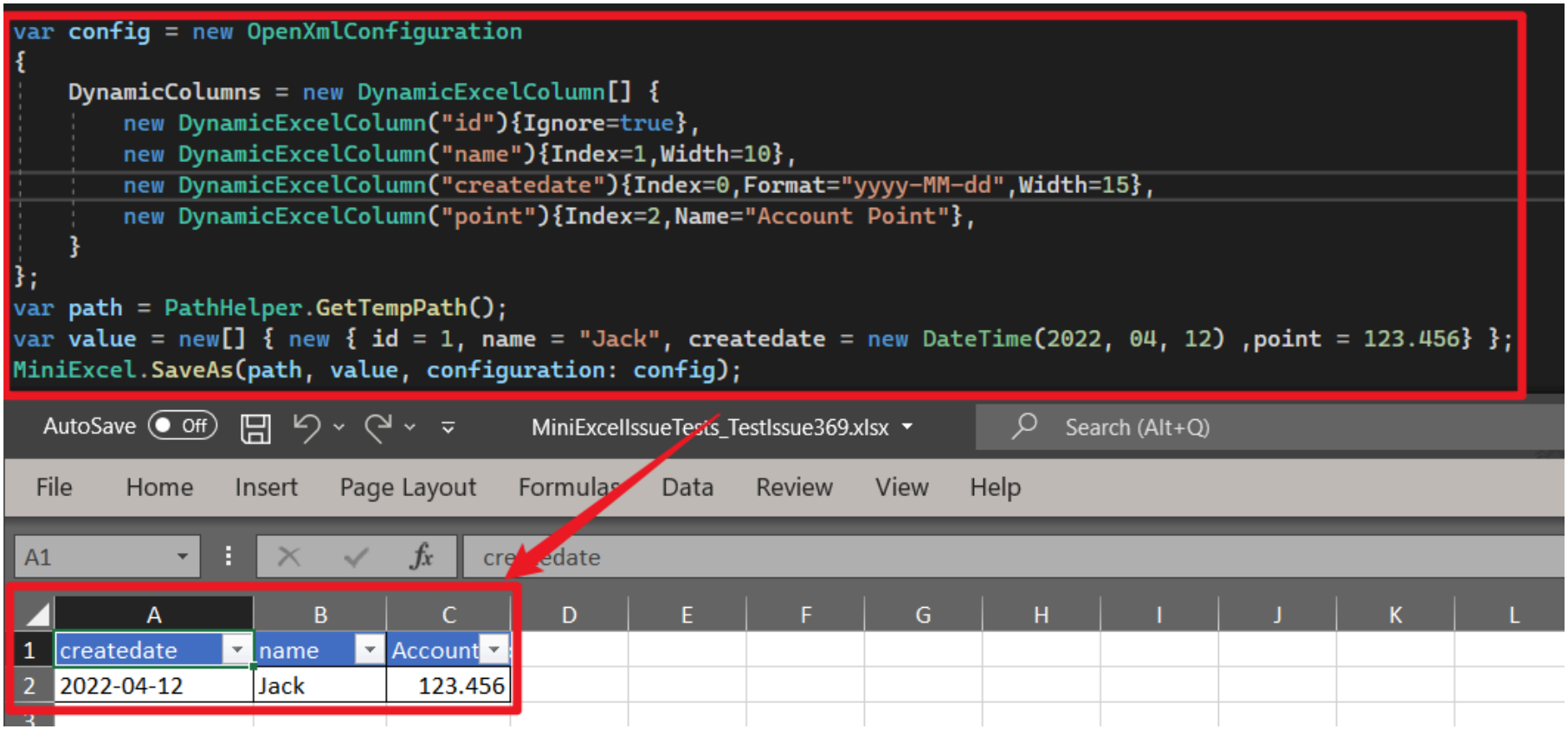
```
public class TestIssueI4ZYUUDto
{
    [ExcelColumn(Name = "ID",Index =0)]
    public string MyProperty { get; set; }
    [ExcelColumn(Name = "CreateDate", Index = 1,Format ="yyyy-MM",Width =100)]
```

```
        public DateTime MyProperty2 { get; set; }
    }
```

7. DynamicColumnAttribute

Since V1.26.0, we can set the attributes of Column dynamically

```
var config = new OpenXmlConfiguration
{
    DynamicColumns = new DynamicExcelColumn[] {
        new DynamicExcelColumn("id"){Ignore=true},
        new DynamicExcelColumn("name"){Index=1,Width=10},
        new DynamicExcelColumn("createdate"){Index=0,Format="yyyy-MM-dd",Width=15},
        new DynamicExcelColumn("point"){Index=2,Name="Account Point"},
    }
};
var path = PathHelper.GetTempPath();
var value = new[] { new { id = 1, name = "Jack", createdate = new DateTime(2022, 04, 12) ,point = 123.456} };
MiniExcel.SaveAs(path, value, configuration: config);
```



8. DynamicSheetAttribute

Since V1.31.4 we can set the attributes of Sheet dynamically. We can set sheet name and state (visibility).

```
var configuration = new OpenXmlConfiguration
{
    DynamicSheets = new DynamicExcelSheet[] {
        new DynamicExcelSheet("usersSheet") { Name = "Users", State = SheetState.Visible },
        new DynamicExcelSheet("departmentSheet") { Name = "Departments", State = SheetState.Hidden }
    }
};

var users = new[] { new { Name = "Jack", Age = 25 }, new { Name = "Mike", Age = 44 } };
var department = new[] { new { ID = "01", Name = "HR" }, new { ID = "02", Name = "IT" } };
var sheets = new Dictionary<string, object>
{
```

```
            ["usersSheet"] = users,
            ["departmentSheet"] = department
        };

        var path = PathHelper.GetTempPath();
        MiniExcel.SaveAs(path, sheets, configuration: configuration);
```

We can also use new attribute ExcelSheetAttribute:

```
[ExcelSheet(Name = "Departments", State = SheetState.Hidden)]
private class DepartmentDto
{
   [ExcelColumn(Name = "ID",Index = 0)]
   public string ID { get; set; }
   [ExcelColumn(Name = "Name",Index = 1)]
   public string Name { get; set; }
}
```

Add, Delete, Update

Add

v1.28.0 support CSV insert N rows data after last row

```
// Origin
{
    var value = new[] {
          new { ID=1,Name ="Jack",InDate=new DateTime(2021,01,03)},
          new { ID=2,Name ="Henry",InDate=new DateTime(2020,05,03)},
    };
    MiniExcel.SaveAs(path, value);
}
// Insert 1 rows after last
{
    var value = new { ID=3,Name = "Mike", InDate = new DateTime(2021, 04, 23) };
    MiniExcel.Insert(path, value);
}
// Insert N rows after last
```

```
{
    var value = new[] {
          new { ID=4,Name ="Frank",InDate=new DateTime(2021,06,07)},
          new { ID=5,Name ="Gloria",InDate=new DateTime(2022,05,03)},
    };
    MiniExcel.Insert(path, value);
}
```

	A	B	C
1	ID	Name	InDate
2	1	Jack	1/3/2021
3	2	Henry	5/3/2020
4	3	Mike	4/23/2021
5	4	Frank	6/7/2021
6	5	Gloria	5/3/2022

v1.37.0 support excel insert a new sheet into an existing workbook

```
// Origin excel
{
    var value = new[] {
          new { ID=1,Name ="Jack",InDate=new DateTime(2021,01,03)},
          new { ID=2,Name ="Henry",InDate=new DateTime(2020,05,03)},
    };
    MiniExcel.SaveAs(path, value, sheetName: "Sheet1");
}
// Insert a new sheet
{
    var value = new { ID=3,Name = "Mike", InDate = new DateTime(2021, 04, 23) };
    MiniExcel.Insert(path, table, sheetName: "Sheet2");
}
```

Delete(waiting)

Update(waiting)

Excel Type Auto Check

- MiniExcel will check whether it is xlsx or csv based on the `file extension` by default, but there may be inaccuracy, please specify it manually.
- Stream cannot be know from which excel, please specify it manually.

```
stream.SaveAs(excelType:ExcelType.CSV);
//or
stream.SaveAs(excelType:ExcelType.XLSX);
//or
stream.Query(excelType:ExcelType.CSV);
//or
stream.Query(excelType:ExcelType.XLSX);
```

CSV

Note

- Default return `string` type, and value will not be converted to numbers or datetime, unless the type is defined by strong typing generic.

Custom separator

The default is `,` as the separator, you can modify the `Seperator` property for customization

```
var config = new MiniExcelLibs.Csv.CsvConfiguration()
{
    Seperator=';'
};
MiniExcel.SaveAs(path, values,configuration: config);
```

Since V1.30.1 support function to custom separator (thanks @hyzx86)

```
var config = new CsvConfiguration()
{
    SplitFn = (row) => Regex.Split(row, $"[\t,](?=(?:[^\"]|\"[^\"]*\")*$)")
        .Select(s => Regex.Replace(s.Replace("\"\"", "\""), "^\"|\"$", "")).ToArray()
};
var rows = MiniExcel.Query(path, configuration: config).ToList();
```

Custom line break

The default is `\r\n` as the newline character, you can modify the `NewLine` property for customization

```
var config = new MiniExcelLibs.Csv.CsvConfiguration()
{
    NewLine='\n'
};
MiniExcel.SaveAs(path, values,configuration: config);
```

Custom coding

- The default encoding is "Detect Encoding From Byte Order Marks" (detectEncodingFromByteOrderMarks: true)
- f you have custom encoding requirements, please modify the StreamReaderFunc / StreamWriterFunc property

```
// Read
var config = new MiniExcelLibs.Csv.CsvConfiguration()
{
    StreamReaderFunc = (stream) => new StreamReader(stream,Encoding.GetEncoding("gb2312"))
};
var rows = MiniExcel.Query(path, true,excelType:ExcelType.CSV,configuration: config);

// Write
var config = new MiniExcelLibs.Csv.CsvConfiguration()
{
    StreamWriterFunc = (stream) => new StreamWriter(stream, Encoding.GetEncoding("gb2312"))
};
MiniExcel.SaveAs(path, value,excelType:ExcelType.CSV, configuration: config);
```

Read empty string as null

By default, empty values are mapped to string.Empty. You can modify this behavior

```
var config = new MiniExcelLibs.Csv.CsvConfiguration()
{
```

```
    ReadEmptyStringAsNull = true
};
```

DataReader

1. GetReader

Since 1.23.0, you can GetDataReader

```
using (var reader = MiniExcel.GetReader(path,true))
{
    while (reader.Read())
    {
        for (int i = 0; i < reader.FieldCount; i++)
        {
            var value = reader.GetValue(i);
        }
    }
}
```

Async

- v0.17.0 support Async (thanks isdaniel (SHIH,BING-SIOU)](https://github.com/isdaniel))

```
public static Task SaveAsAsync(string path, object value, bool printHeader = true, string sheetName = "Sheet1", ExcelType excel
public static Task SaveAsAsync(this Stream stream, object value, bool printHeader = true, string sheetName = "Sheet1", ExcelTyp
public static Task<IEnumerable<dynamic>> QueryAsync(string path, bool useHeaderRow = false, string sheetName = null, ExcelType
public static Task<IEnumerable<T>> QueryAsync<T>(this Stream stream, string sheetName = null, ExcelType excelType = ExcelType.U
public static Task<IEnumerable<T>> QueryAsync<T>(string path, string sheetName = null, ExcelType excelType = ExcelType.UNKNOWN,
public static Task<IEnumerable<IDictionary<string, object>>> QueryAsync(this Stream stream, bool useHeaderRow = false, string sh
public static Task SaveAsByTemplateAsync(this Stream stream, string templatePath, object value)
public static Task SaveAsByTemplateAsync(this Stream stream, byte[] templateBytes, object value)
public static Task SaveAsByTemplateAsync(string path, string templatePath, object value)
public static Task SaveAsByTemplateAsync(string path, byte[] templateBytes, object value)
public static Task<DataTable> QueryAsDataTableAsync(string path, bool useHeaderRow = true, string sheetName = null, ExcelType e
```

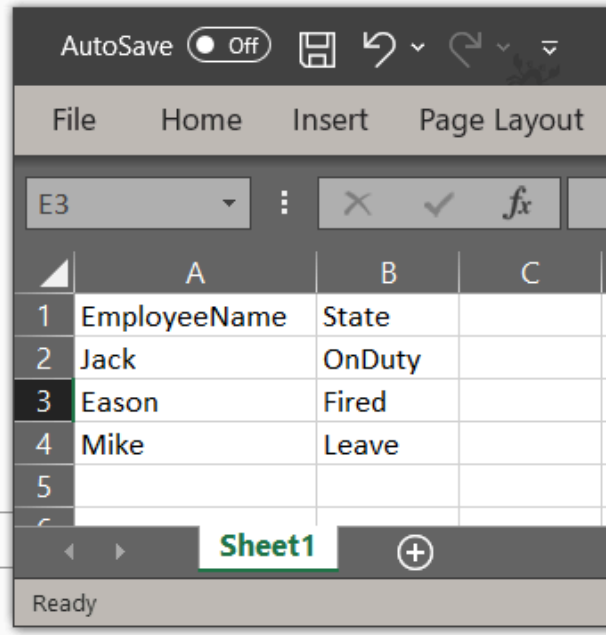
- v1.25.0 support `cancellationToken`。

Others

1. Enum

Be sure excel & property name same, system will auto mapping (case insensitive)

```
1  void Main()
2  {
3      var path = @"C:\Users\Wei\Downloads\Employees.xlsx";
4      var rows = MiniExcel.Query<Employee>(path).ToList();
5      Console.WriteLine(rows);
6  }
7
8  public class Employee
9  {
10     public string EmployeeName { get; set; }
11     public WorkState State { get; set; }
12 }
13
14 public enum WorkState
15 {
16     OnDuty,
17     Leave,
18     Fired
19 }
```



Results λ SQL IL Tree

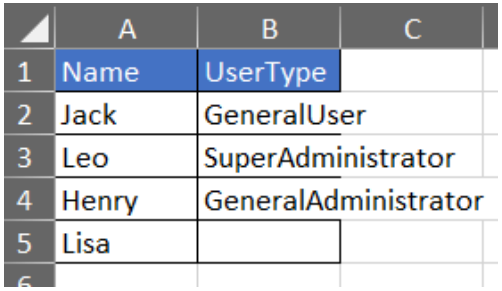
List<Employee> (3 items)

EmployeeName	State
Jack	OnDuty
Eason	Fired
Mike	Leave

Since V0.18.0 support Enum Description

```
public class Dto
{
    public string Name { get; set; }
    public I49RYZUserType UserType { get; set; }
}

public enum Type
{
    [Description("General User")]
    V1,
    [Description("General Administrator")]
    V2,
    [Description("Super Administrator")]
    V3
}
```



	A	B	C
1	Name	UserType	
2	Jack	GeneralUser	
3	Leo	SuperAdministrator	
4	Henry	GeneralAdministrator	
5	Lisa		

Since 1.30.0 version support excel Description to Enum , thanks @KaneLeung

2. Convert CSV to XLSX or Convert XLSX to CSV

```
MiniExcel.ConvertXlsxToCsv(xlsxPath, csvPath);
MiniExcel.ConvertXlsxToCsv(xlsxStream, csvStream);
MiniExcel.ConvertCsvToXlsx(csvPath, xlsxPath);
MiniExcel.ConvertCsvToXlsx(csvStream, xlsxStream);
```

```
using (var excelStream = new FileStream(path: filePath, FileMode.Open, FileAccess.Read))
using (var csvStream = new MemoryStream())
{
```

```
    MiniExcel.ConvertXlsxToCsv(excelStream, csvStream);
}
```

3. Custom CultureInfo

Since 1.22.0, you can custom CultureInfo like below, system default `CultureInfo.InvariantCulture` .

```
var config = new CsvConfiguration()
{
    Culture = new CultureInfo("fr-FR"),
};
MiniExcel.SaveAs(path, value, configuration: config);

// or
MiniExcel.Query(path, configuration: config);
```

4. Custom Buffer Size

```
    public abstract class Configuration : IConfiguration
    {
        public int BufferSize { get; set; } = 1024 * 512;
    }
```

5. FastMode

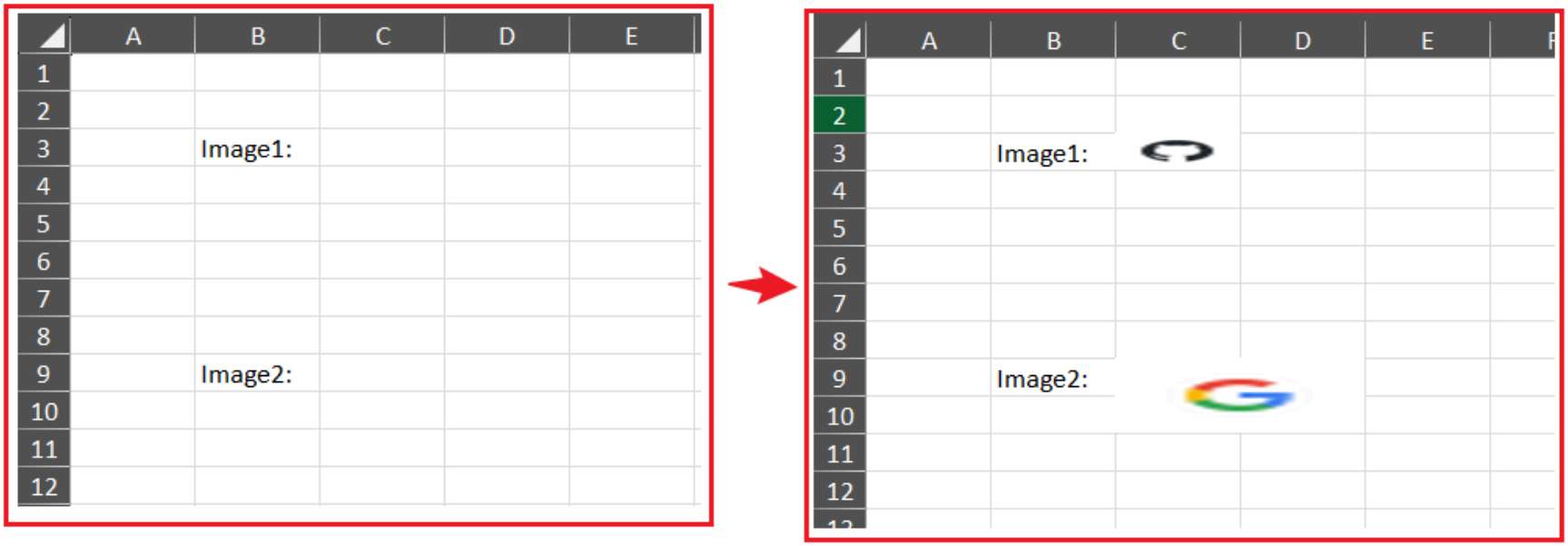
System will not control memory, but you can get faster save speed.

```
var config = new OpenXmlConfiguration() { FastMode = true };
MiniExcel.SaveAs(path, reader,configuration:config);
```

6. Batch Add Image (MiniExcel.AddPicture)

```
var images = new[]
{
    new MiniExcelPicture
```

```
    {
        ImageBytes = File.ReadAllBytes(PathHelper.GetFile("images/github_logo.png")),
        SheetName = null, // default null is first sheet
        CellAddress = "C3", // required
    },
    new MiniExcelPicture
    {
        ImageBytes = File.ReadAllBytes(PathHelper.GetFile("images/google_logo.png")),
        PictureType = "image/png", // default PictureType = image/png
        SheetName = "Demo",
        CellAddress = "C9", // required
        WidthPx = 100,
        HeightPx = 100,
    },
};
MiniExcel.AddPicture(path, images);
```



7. Get Sheets Dimension

```
var dim = MiniExcel.GetSheetsDimensions(path);
```

Examples:

1. SQLite & Dapper `Large Size File` SQL Insert Avoid OOM

note : please don't call ToList/ToArray methods after Query, it'll load all data into memory

```
using (var connection = new SQLiteConnection(connectionString))
{
    connection.Open();
    using (var transaction = connection.BeginTransaction())
    using (var stream = File.OpenRead(path))
    {
       var rows = stream.Query();
       foreach (var row in rows)
             connection.Execute("insert into T (A,B) values (@A,@B)", new { row.A, row.B }, transaction: transaction);
       transaction.Commit();
    }
}
```

performance:

```
    var path = @"D:\git\MiniExcel\samples\xlsx\Test1,000,000x10\Test1,000,000x10.xlsx";
    var tempSqlitePath = Path.Combine(Path.GetTempPath(), $"{Guid.NewGuid()}.db");
    var connectionString = $"Data Source={tempSqlitePath};Version=3;";
    SQLiteConnection.CreateFile(tempSqlitePath);

    using (var connection = new SQLiteConnection(connectionString))
    {
        connection.Execute(@"create table T (A varchar(20),B varchar(20));");
    }

    Stopwatch sw = new Stopwatch();
    sw.Start();
    Console.WriteLine("start memory usage: " + System.Diagnostics.Process.GetCurrentProcess().WorkingSet64 / (1024 * 1024) + $"MB");

    using (var connection = new SQLiteConnection(connectionString))
    {
        connection.Open();
        using (var transaction = connection.BeginTransaction())
        using (var stream = File.OpenRead(path))
        {
            var rows = stream.Query();
            foreach (var row in rows)
                connection.Execute("insert into T (A,B) values (@A,@B)", new { row.A, row.B }, transaction: transaction);
            transaction.Commit();
        }

        Console.WriteLine("end memory usage: " + System.Diagnostics.Process.GetCurrentProcess().WorkingSet64 / (1024 * 1024) + $"MB & run time : {sw.ElapsedMilliseconds}ms");
    }

    using (var connection = new SQLiteConnection(connectionString))
    {
        var count = connection.ExecuteScalar<int>("select count(*) from T");
        Console.WriteLine($"count : {count}");
    }
}
```

Results λ SQL IL Tree

```
start memory usage: 61MB
end memory usage: 70MB & run time : 30378ms
count : 1000000
```

2. ASP.NET Core 3.1 or MVC 5 Download/Upload Excel Xlsx API Demo Try it

```
public class ApiController : Controller
{
    public IActionResult Index()
    {
        return new ContentResult
        {
            ContentType = "text/html",
            StatusCode = (int)HttpStatusCode.OK,
            Content = @"<html><body>
```

```
<a href='api/DownloadExcel'>DownloadExcel</a><br>
<a href='api/DownloadExcelFromTemplatePath'>DownloadExcelFromTemplatePath</a><br>
<a href='api/DownloadExcelFromTemplateBytes'>DownloadExcelFromTemplateBytes</a><br>
<p>Upload Excel</p>
<form method='post' enctype='multipart/form-data' action='/api/uploadexcel'>
    <input type='file' name='excel'> <br>
    <input type='submit' >
</form>
</body></html>"
        };
    }

    public IActionResult DownloadExcel()
    {
        var values = new[] {
            new { Column1 = "MiniExcel", Column2 = 1 },
            new { Column1 = "Github", Column2 = 2}
        };
        var memoryStream = new MemoryStream();
        memoryStream.SaveAs(values);
        memoryStream.Seek(0, SeekOrigin.Begin);
        return new FileStreamResult(memoryStream, "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
        {
            FileDownloadName = "demo.xlsx"
        };
    }

    public IActionResult DownloadExcelFromTemplatePath()
    {
        string templatePath = "TestTemplateComplex.xlsx";

        Dictionary<string, object> value = new Dictionary<string, object>()
        {
            ["title"] = "FooCompany",
            ["managers"] = new[] {
                new {name="Jack",department="HR"},
                new {name="Loan",department="IT"}
            },
            ["employees"] = new[] {
                new {name="Wade",department="HR"},
                new {name="Felix",department="HR"},
                new {name="Eric",department="IT"},
```

```
            new {name="Keaton",department="IT"}
        }
    };

    MemoryStream memoryStream = new MemoryStream();
    memoryStream.SaveAsByTemplate(templatePath, value);
    memoryStream.Seek(0, SeekOrigin.Begin);
    return new FileStreamResult(memoryStream, "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
    {
        FileDownloadName = "demo.xlsx"
    };
}

private static Dictionary<string, Byte[]> TemplateBytesCache = new Dictionary<string, byte[]>();

static ApiController()
{
    string templatePath = "TestTemplateComplex.xlsx";
    byte[] bytes = System.IO.File.ReadAllBytes(templatePath);
    TemplateBytesCache.Add(templatePath, bytes);
}

public IActionResult DownloadExcelFromTemplateBytes()
{
    byte[] bytes = TemplateBytesCache["TestTemplateComplex.xlsx"];

    Dictionary<string, object> value = new Dictionary<string, object>()
    {
        ["title"] = "FooCompany",
        ["managers"] = new[] {
            new {name="Jack",department="HR"},
            new {name="Loan",department="IT"}
        },
        ["employees"] = new[] {
            new {name="Wade",department="HR"},
            new {name="Felix",department="HR"},
            new {name="Eric",department="IT"},
            new {name="Keaton",department="IT"}
        }
    };

    MemoryStream memoryStream = new MemoryStream();
```

```
            memoryStream.SaveAsByTemplate(bytes, value);
            memoryStream.Seek(0, SeekOrigin.Begin);
            return new FileStreamResult(memoryStream, "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
            {
                FileDownloadName = "demo.xlsx"
            };
        }

        public IActionResult UploadExcel(IFormFile excel)
        {
            var stream = new MemoryStream();
            excel.CopyTo(stream);

            foreach (var item in stream.Query(true))
            {
                // do your logic etc.
            }

            return Ok("File uploaded successfully");
        }
    }
```

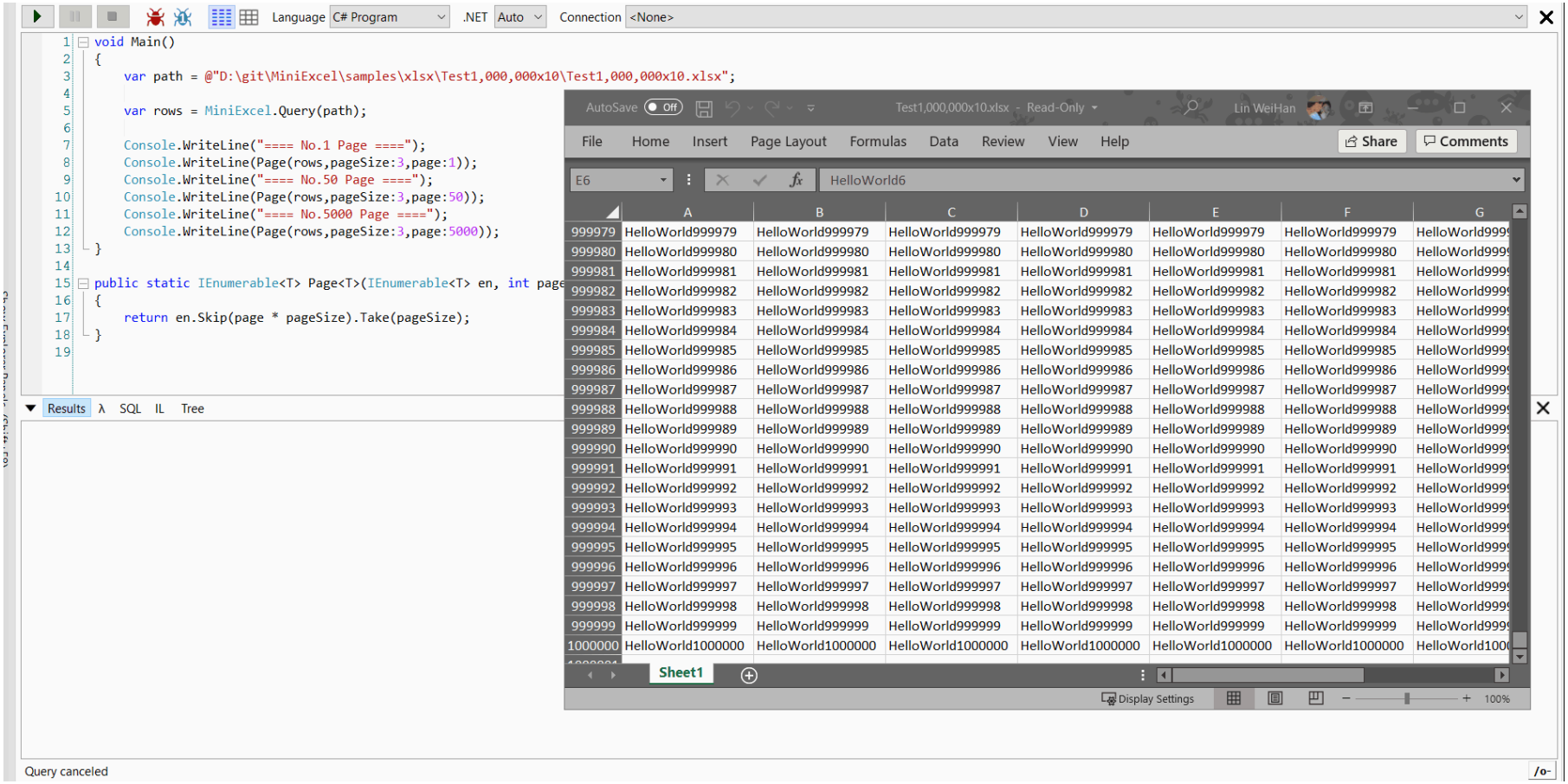
3. Paging Query

```
    void Main()
    {
        var rows = MiniExcel.Query(path);

        Console.WriteLine("==== No.1 Page ====");
        Console.WriteLine(Page(rows,pageSize:3,page:1));
        Console.WriteLine("==== No.50 Page ====");
        Console.WriteLine(Page(rows,pageSize:3,page:50));
        Console.WriteLine("==== No.5000 Page ====");
        Console.WriteLine(Page(rows,pageSize:3,page:5000));
    }

    public static IEnumerable<T> Page<T>(IEnumerable<T> en, int pageSize, int page)
    {
```

```
    return en.Skip(page * pageSize).Take(pageSize);
}
```



4. WebForm export Excel by memorystream

```
var fileName = "Demo.xlsx";
var sheetName = "Sheet1";
HttpResponse response = HttpContext.Current.Response;
response.Clear();
response.ContentType = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet";
response.AddHeader("Content-Disposition", $"attachment;filename=\"{fileName}\"");
var values = new[] {
    new { Column1 = "MiniExcel", Column2 = 1 },
```

```
    new { Column1 = "Github", Column2 = 2}
};
var memoryStream = new MemoryStream();
memoryStream.SaveAs(values, sheetName: sheetName);
memoryStream.Seek(0, SeekOrigin.Begin);
memoryStream.CopyTo(Response.OutputStream);
response.End();
```

5. Dynamic i18n multi-language and role authority management

Like the example, create a method to handle i18n and permission management, and use `yield return to return IEnumerable<Dictionary<string, object>>` to achieve dynamic and low-memory processing effects

```
void Main()
{
    var value = new Order[] {
        new Order(){OrderNo = "SO01",CustomerID="C001",ProductID="P001",Qty=100,Amt=500},
        new Order(){OrderNo = "SO02",CustomerID="C002",ProductID="P002",Qty=300,Amt=400},
    };

    Console.WriteLine("en-Us and Sales role");
    {
        var path = Path.GetTempPath() + Guid.NewGuid() + ".xlsx";
        var lang = "en-US";
        var role = "Sales";
        MiniExcel.SaveAs(path, GetOrders(lang, role, value));
        MiniExcel.Query(path, true).Dump();
    }

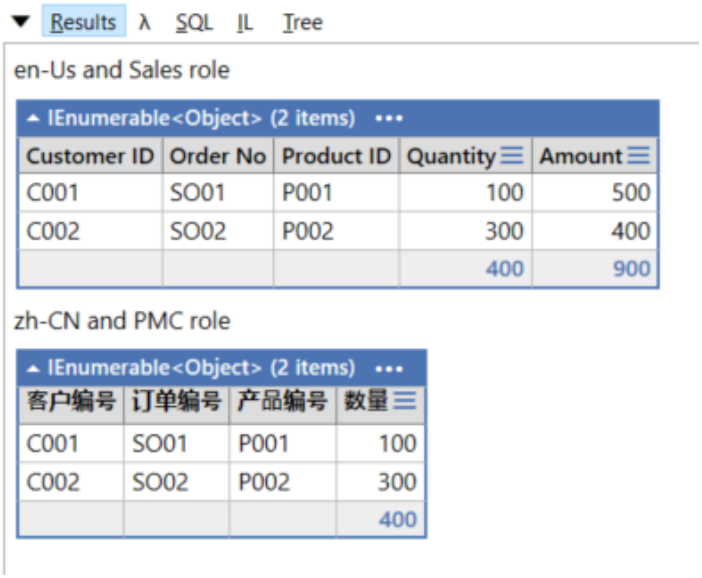
    Console.WriteLine("zh-CN and PMC role");
    {
        var path = Path.GetTempPath() + Guid.NewGuid() + ".xlsx";
        var lang = "zh-CN";
        var role = "PMC";
        MiniExcel.SaveAs(path, GetOrders(lang, role, value));
        MiniExcel.Query(path, true).Dump();
    }
}

private IEnumerable<Dictionary<string, object>> GetOrders(string lang, string role, Order[] orders)
```

```
{
    foreach (var order in orders)
    {
        var newOrder = new Dictionary<string, object>();

        if (lang == "zh-CN")
        {
            newOrder.Add("客户编号", order.CustomerID);
            newOrder.Add("订单编号", order.OrderNo);
            newOrder.Add("产品编号", order.ProductID);
            newOrder.Add("数量", order.Qty);
            if (role == "Sales")
                newOrder.Add("价格", order.Amt);
            yield return newOrder;
        }
        else if (lang == "en-US")
        {
            newOrder.Add("Customer ID", order.CustomerID);
            newOrder.Add("Order No", order.OrderNo);
            newOrder.Add("Product ID", order.ProductID);
            newOrder.Add("Quantity", order.Qty);
            if (role == "Sales")
                newOrder.Add("Amount", order.Amt);
            yield return newOrder;
        }
        else
        {
            throw new InvalidDataException($"lang {lang} wrong");
        }
    }
}

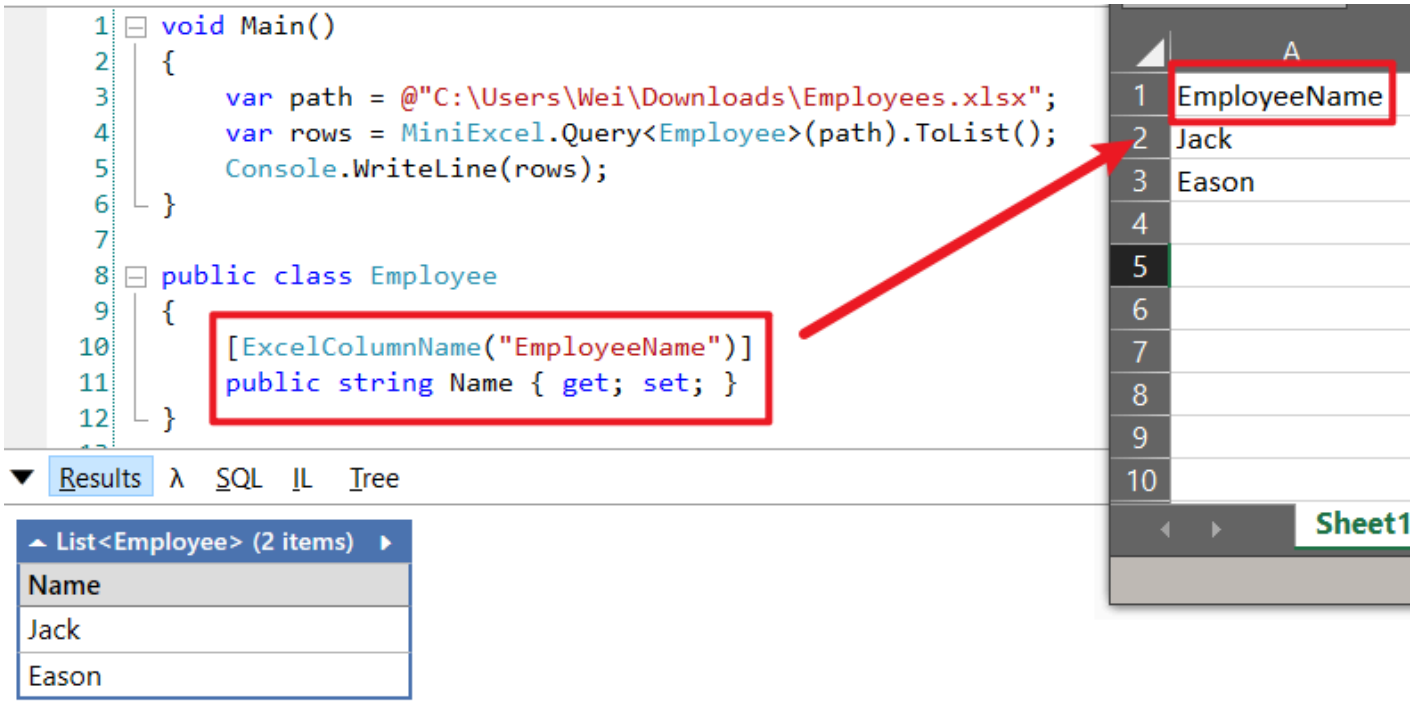
public class Order
{
    public string OrderNo { get; set; }
    public string CustomerID { get; set; }
    public decimal Qty { get; set; }
    public string ProductID { get; set; }
    public decimal Amt { get; set; }
}
```



FAQ

Q: Excel header title not equal class property name, how to mapping?

A. Please use ExcelColumnName attribute



Q. How to query or export multiple-sheets?

A. `GetSheetNames` method with Query sheetName parameter.

```
var sheets = MiniExcel.GetSheetNames(path);
foreach (var sheet in sheets)
{
    Console.WriteLine($"sheet name : {sheet} ");
    var rows = MiniExcel.Query(path,useHeaderRow:true,sheetName:sheet);
    Console.WriteLine(rows);
}
```

```
void Main()
{
    var path = @"C:\Users\Wei\Downloads\multiple_sheets.xlsx";
    var sheets = MiniExcel.GetSheetNames(path);
    foreach (var sheet in sheets)
    {
        Console.WriteLine($"sheet name : {sheet} ");
        var rows = MiniExcel.Query(path,useHeaderRow:true,sheetName:sheet);
        Console.WriteLine(rows);
    }
}
```

Results λ SQL IL Tree

sheet name : Sheet1

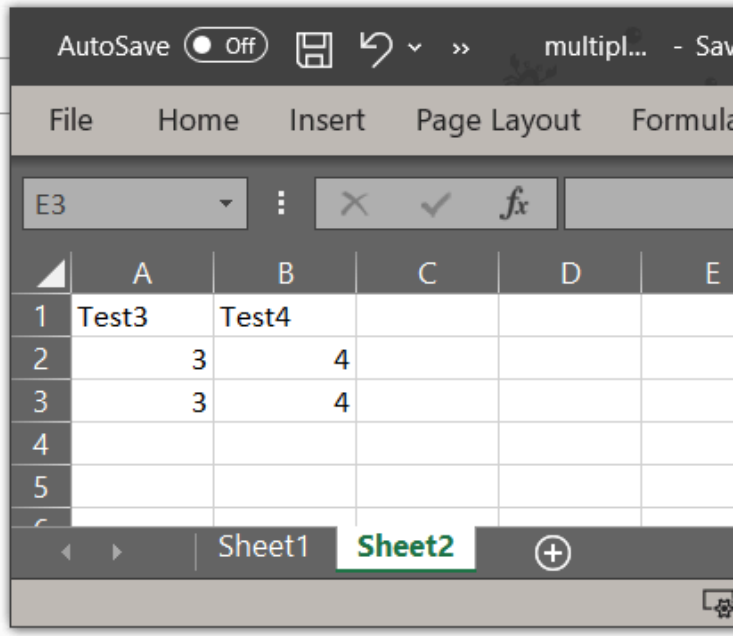
IEnumerable<Object> (2 items)

Test1	Test2
1	2
1	2
2	4

sheet name : Sheet2

IEnumerable<Object> (2 items)

Test3	Test4
3	4
3	4
6	8



Q. How to query or export information about sheet visibility?

A. `GetSheetInformations` method.

```
var sheets = MiniExcel.GetSheetInformations(path);
foreach (var sheetInfo in sheets)
{
```